ETH zürich

QUDEV

Realizing Hybrid Quantum Neural Networks on a 9-qubit Surface Code to Recognize Topological Quantum Phases

Master's Thesis

Shengpu Wang

wangshen@student.ethz.ch

Quantum Device Lab

Department of Physics, D-PHYS

ETH Zürich

Supervisors:

Colin Scarato

Principal Investigator:

Prof. Dr. Andreas Wallraff

January 6, 2025

Abstract

In this thesis, we explore the training and application of hybrid neural networks (HNNs) as a novel approach for topological phase recognition. Existing classical methods, such as classical shadows, face intrinsic limitations in detecting topological phases due to the non-local nature of topological phases, particularly in two-dimensional systems. These methods often struggle to capture global quantum correlations essential for the topological phase recognition. To address these challenges, quantum machine learning models, such as the HNNs explored in this work, are proposed as an alternative. Prior research has demonstrated that quantum circuits designed for this purpose are robust against noise and well-suited for implementation on noisy intermediate-scale quantum (NISQ) devices, making them a promising tool for overcoming the limitations of classical approaches.

This work begins with implementing and training an HNN to recognize topological phases in perturbed surface code ground states using a 4-qubit quantum processor. The HNN uses a parameterized quantum circuit with classical post-processing, enabling efficient phase recognition through a hybrid quantum-classical optimization process. The quantum gate parameters are optimized on a classical computer to enhance the performance. The HNNs are then tested for two primary use cases: phase boundary detection and single-shot classification, demonstrating their capability in identifying phase transitions and distinguishing topological states from trivial states, respectively. Building on the conclusions gained from the 4-qubit experiments, we scale the HNN experiment to a 9-qubit system to examine the scalability of the HNN to higher-dimensional quantum systems.

The findings of this thesis demonstrate the potential of HNNs for efficient quantum phase recognition and broader applications for complex, large-scale quantum systems. This work provides a foundation for future studies on the application of HNNs to practical solid state physics problems, paving the way for more advanced quantum neural network architectures.

Contents

1 Introduction **4**
- 1.1 Motivation of quantum machine learning for phase transition 4
  - 1.1.1 Classical methods for topological phase recognition 4
  - 1.1.2 Quantum circuits for topological phase recognition 5
- 1.2 Quantum circuit training in hardware 6

2 Concept of the Experiment **9**
- 2.1 Phase transitions in surface code ground states 9
- 2.2 Hybrid Neural Network 12
  - 2.2.1 Overview and Circuit Diagram 12
  - 2.2.2 Training set, output and cost function 13
  - 2.2.3 Gradient descent optimizer 16
- 2.3 Experiment setup 17
  - 2.3.1 Software structure 17
  - 2.3.2 Qubit and gate calibration 18

3 Training Experiment on a 4-qubit System **20**
- 3.1 State preparation 20
- 3.2 Training experiment 21
- 3.3 Concept of the verification experiments 23
  - 3.3.1 Use case: phase boundary detection 23
  - 3.3.2 Use case: single-shot classification 24
- 3.4 Experiment data analysis 26
  - 3.4.1 Comparison of initialization conditions 26
  - 3.4.2 Comparison of training sets 28

4 Comparison with a 9-qubit Experiment **33**
- 4.1 State preparation 33
- 4.2 Experiment data analysis 34
  - 4.2.1 Comparison of initialization conditions 34
  - 4.2.2 Comparison of training sets 36

Conclusion **39**

Chapter 1

Introduction

Topological phase recognition is a crucial area in condensed matter physics, focusing on identifying and characterizing phases of matter that are defined by their global, non-local properties rather than local order parameters. Recognizing these phases is challenging, especially in two-dimensional systems [1]. Leveraging quantum techniques, such as quantum circuits and hybrid neural networks, offers a promising approach to efficiently detect and classify topological phases.

1.1 Motivation of quantum machine learning for phase transition

Unlike trivial phases which can be detected by measuring local observables, topological phases cannot be characterized by conventional local order parameters [2]. The topological phase is a global property of the system. The detection of the topological phase requires the information about the entire system's quantum state. The inherent non-locality poses a significant challenge for detecting the topological phase theoretically and experimentally [3]. Theoretically, the exact calculation or simulation of the quantum many-body system is extremely complex and often requires complicated computational techniques, such as quantum Monte Carlo. In the experiment, existing quantum measurement methods are typically based on local observables. Probing global features of a quantum state requires complex experiment set-ups or pre-processing. The challenges above make the detection of the topological phase a demanding task. In the next subsection, existing classical methods for topological phase detection and their limitations are provided as background information. The limitations then motivate the use of quantum circuits in topological phase detection to achieve higher accuracy and efficiency.

1.1.1 Classical methods for topological phase recognition

The detection of symmetry-protected topological phase in one-dimensional systems has been investigated in [2]. In this work, string order parameters were introduced to distinguish topological phases from trivial phases by identifying the hidden symmetries. Specifically, it was

demonstrated that the string order parameters can successfully detect the $Z_2 \times Z_2$ symmetry-protected topological phase in the spin-1 Heisenberg chain. However, a notable limitation of the string order parameter method is that it is only applied to the matrix product states (MPS) representation of quantum states. Besides, the high computational cost of the MPS representation calls the applicability of this method to high-dimensional systems into question.

It is demonstrated in the following two papers that classical machine learning models can effectively classify non-conventional quantum phases, by leveraging data from quantum experiments or simulations even when string order parameter methods fail. In [1], the authors showcased how classical machine learning methods can be effectively applied to topological phase detection, offering a new approach to investigate phase transitions. Classical neural networks were used to process Monte-Carlo simulated data and successfully detected various phases of matter. The fully-connected neural network demonstrated its capability to identify the topological phase in the Ising ferromagnetic model, while the convolutional neural network was applied to the Ising lattice gauge model, where it also successfully detected the presence of a topological phase. The results highlight the strength of machine learning methods in recognizing hidden patterns and non-local features in high-dimensional data. The state-of-the-art work on classical methods for quantum phase recognition is presented in [4]. A key innovation in this work is the use of classical shadows, a compact classical representation of quantum states obtained from measurements in random bases. By utilizing this representation, the authors trained supervised machine learning models for ground-state energy prediction and kernel principal component analysis (PCA) models for phase recognition. However, the classical neural networks require an exponential sampling cost near phase boundaries, where distinguishing features are harder to extract.

1.1.2 Quantum circuits for topological phase recognition

Due to the limited performance of classical methods in recognizing topological phases, particularly their inability to efficiently capture distinguishing features near the phase boundary, researchers have turned to quantum circuits as a promising alternative. Quantum circuits can naturally process quantum data and exploit the inherent properties of quantum states, allowing them to recognize topological phases more efficiently. Quantum convolutional neural networks (QCNN) are introduced in [5] as a quantum analog of classical convolutional neural networks, specifically designed for recognizing quantum phases in many-body systems. QCNNs were developed to address the challenges posed by traditional methods, such as string order parameters, which often require extensive measurements and computational resources to identify the phases of matter. A key advantage of the proposed QCNN architecture is its computational efficiency: for an N-qubit input state, the QCNN requires only $O(\log N)$ parameters, making it highly scalable and well-suited for implementation on near-term quantum devices with limited resources. This efficiency stems from the hierarchical structure of the QCNN, which mirrors the layered filtering and pooling operations of classical convolutional networks, enabling it to process quantum states efficiently while preserving relevant phase features. In numerical simulations, the trained QCNN successfully distinguished different quantum phases with significantly lower sample complexity compared

to conventional methods based on string order parameters.

[6] is the first experimental realization of the proposed QCNN for recognizing the topological phase in the cluster-Ising Hamiltonian ground states. The QCNN architecture is inspired by classical convolutional neural networks, consisting of convolutional layers, pooling layers, and a fully-connected layer, which work together to extract relevant features for phase recognition. To enhance efficiency and reduce the number of quantum gates, the pooling and fully-connected layers were partially implemented in a classical processing unit. This hybrid quantum-classical approach not only optimizes resource utilization but also enables the QCNN to outperform traditional methods, such as direct measurements of string order parameters, in identifying the topological phase. Despite the success of the experiment, a key limitation remains: the gate parameters were optimized theoretically and not trained on the quantum hardware. This highlights the need for future developments in experimental training methods to fully realize the potential of QCNNs.

[7] presents the most recent development of QCNN models for detecting phase transitions in the toric code model, a system exhibiting Z_2-topological order. Through numerical simulations, the QCNN was demonstrated to successfully identify the topological phase, with its output converging to a clear distinction between phases. Notably, the accuracy of the phase detection improves as the number of pulling layers in the QCNN increases, analogous to classical convolutional neural networks. Despite its success, the QCNN, similar to prior works such as [6], was designed based on an existing theoretical model rather than being optimized through a training process.

1.2 Quantum circuit training in hardware

It is essential to highlight similar experimental examples of quantum circuit training on hardware, even though they may not be directly comparable to the HNN experiment presented in this work. While the specific objectives and methodologies may differ, such experiments showcase the progress and challenges in practical settings.

[8] used a variational quantum eigensolver (VQE) combined with subspace diagonalization and classical post-processing on a 10-qubit trapped-ion quantum processor to calculate the ground state energy of the Fe(III)-NTA metal chelate. The circuit ansatz is parameterized by 20 independent parameters. The ground state energies predicted by the trained VQE achieve chemical accuracy. [9] reports the experimental training of deep quantum neural networks for simulating quantum channels and predicting the ground state energies of the hydrogen molecule using a 6-qubit superconducting quantum processor. The network employs a layer-by-layer quantum perceptron structure. In this study, the forward propagation of the neural network was successfully implemented on the quantum processor. However, the backward process, which involves gradient evaluation and parameter updates, was performed on a classical computer. While this and the previous approach demonstrated the potential of quantum neural networks for solving quantum chemistry problems, a key limitation was that the studies above relied solely on VQE-type state preparation. VQEs are effective for

specific tasks like energy minimization but do not fully align with our goal of recognizing non-trivial quantum phases of matter.

In [3], quantum neural networks (QNN) are used to classify ergodic and localized quantum phases. The ergodic phase refers to a quantum phase of which the local observables are indistinguishable from those of a thermal ensemble [10]. Implemented on a 61-qubit processor, the QNN realized phase classification by pre-processing input quantum states with the quantum neural network. This method required measurements of only one qubit to achieve efficient classification, significantly reducing the measurement overhead compared to traditional methods. However, a limitation of this work lies in its application to an artificially constructed task, where the ergodic phase is represented by trivial states obfuscated by random flux pulses, rather than addressing more physically meaningful or experimentally challenging systems.

A novel quantum end-to-end learning framework that leverages pulse-based quantum neural networks was introduced in [11] for the classification of classical data. The framework is designed to bypass traditional quantum circuit design by directly parameterizing the quantum ansatz through manipulable control pulses, making it inherently gate-free. This approach is hardware-friendly, as it allows for the full utilization of limited quantum resources while avoiding the complexities of circuit calibration. The study achieved a classification accuracy of approximately 98% for binary classification tasks using two qubits and 89% for four-class classification tasks with three qubits. However, the framework was applied exclusively to classical data, leaving its applicability to inherent quantum data and problems unexplored.

[12] explores the implementation of hybrid quantum machine learning methods on a 4-qubit processor to address supervised learning tasks, including multi-class classification and image recognition. The proposed quantum model leverages parameterized single-qubit gates and iSWAP gates to process data effectively, utilizing the concept of data reuploading to encode classical information multiple times into the quantum circuit. But the trained hybrid quantum model did not outperform classical methods in terms of accuracy, and the training process was considerably slower than classical counterparts due to the limited repetition rate of the quantum hardware.

The use of quantum kernel estimation methods to improve the efficiency of the training of machine learning models was investigated in [13]. This was done by utilizing the group structure of data. By incorporating symmetry properties into the kernel design, the quantum kernel estimation algorithm can exploit this structure to improve performance and reduce computational overhead. The researchers implemented this method on a 27-qubit processor, demonstrating its effectiveness in solving an artificially constructed classification problem. But the applicability of quantum kernels to practical problems remains an open question, as this study only focused on a designed classification task.

[14] introduces a hybrid quantum machine learning model that leverages quantum embeddings to enhance learning performance. The proposed framework uses a quantum embedding space, where classical data is encoded into quantum states, to better represent and separate

data for supervised learning tasks. In the experimental implementation, the quantum embedding was trained to maximize the trace distance between quantum states in the quantum feature space, with its accuracy tested on the MNIST dataset. But the parameters of the quantum embedding were not optimized directly during the training process; instead, they were determined by the output of a classical neural network.

[15] presents a state-of-the-art implementation of a VQE experiment aimed at improving the accuracy of quantum simulations of electron systems by focusing on paired electron configurations. The VQE ansatz was designed with a structure based on nearest-neighbor couplings, where each layer consists of single-qubit rotation gates followed by SWAP gates to facilitate interactions between qubit pairs. Only the single-qubit rotation gates were parameterized, with their parameters optimized by a classical optimizer to minimize the energy of the quantum state prepared by the circuit. In this master thesis work, all the quantum gate parameters are tunable and optimized in the training experiment.

Chapter 2

Concept of the Experiment

The HNN experiment is designed to explore phase transitions in the ground states of the surface code. The HNN integrates a parameterized quantum circuit (PQC) with a classical optimizer for iterative optimization, creating a hybrid structure. The training process utilizes a training set, including both topological and trivial quantum states, and is guided by a cost function that quantifies the difference between the HNN's predictions and the true phase labels.

To carry out the HNN experiment, qubit and gate calibrations are indispensable to guarantee the quality of the experimental data. Information about qubit and gate properties are provided in this chapter as well.

2.1 Phase transitions in surface code ground states

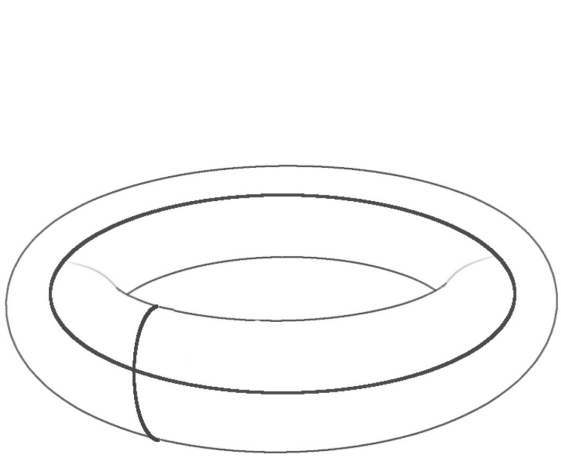
The toric code model is a paradigmatic example of a quantum system exhibiting topological features and robustness against local perturbations. The toric code model can be driven through a topological phase transition by introducing a uniform magnetic field. This transition marks a shift between distinct quantum phases, including a topological phase and trivial polarized phases, depending on the field's strength and orientation [18].

[16] investigates the phase transitions in the toric code model subjected to a parallel magnetic field as in Eq. 2.1, providing a comprehensive analysis of the system's behavior under such conditions.

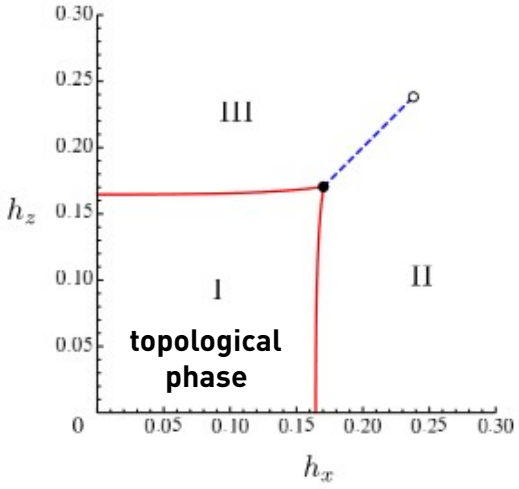
$$H = H_{\text{toric}} - h_x \sum_i \sigma_i^x - h_z \sum_i \sigma_i^z \tag{2.1}$$

The research explores both first-order and second-order phase transitions. The resulting phase diagram, shown in Figure 2.1b, includes three distinct phases: a topological phase and two polarized (trivial) phases, separated by second-order transition lines (depicted in red) and a first-order transition line (depicted in blue).

In this work, the surface code model is studied because of its experimental importance. The surface code model is closely related to the toric code model. Both are based on spin-1/2

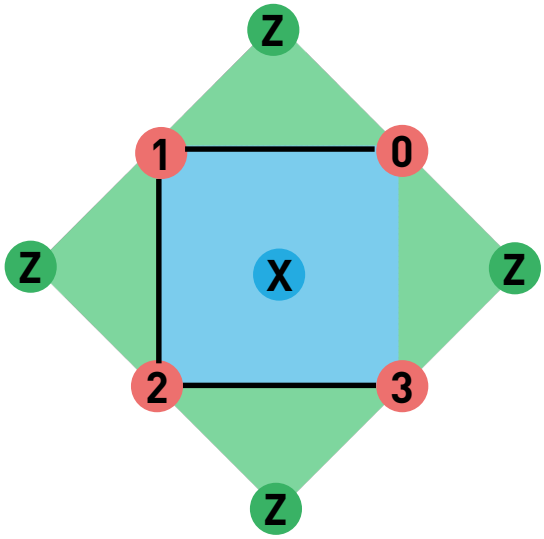


(a) A torus, on which the toric code is defined

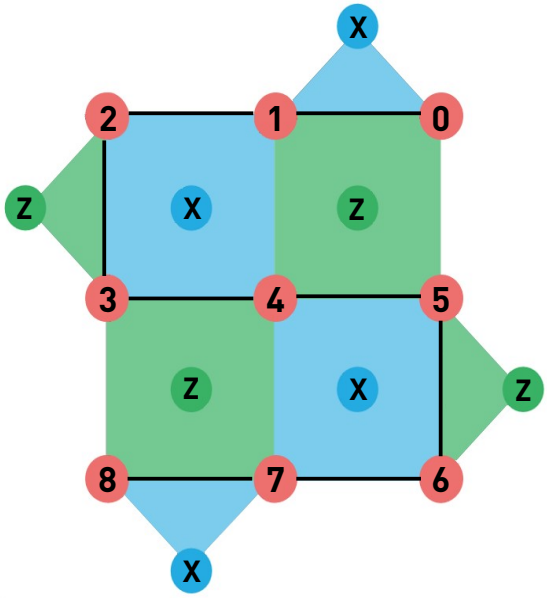


(b) Toric code phase transition

Figure 2.1: Diagram of the toric code and its phase transitions in a uniform magnetic field [16]



(a) 4-qubit stabilizers geometry



(b) 9-qubit stabilizers geometry

Figure 2.2: Stabilizers geometry

systems and are defined using X and Z stabilizer operators. The primary difference lies in the boundary conditions, as the toric code model is defined on the surface of a torus with periodic boundary conditions, while the surface code model is defined on a lattice with open boundary conditions. The open boundary condition results in a planar geometry, which is well-suited for physical layouts on experimental devices [19]. The surface code models considered in this work are 4- and 9-qubit systems, as illustrated in Figure 2.2.

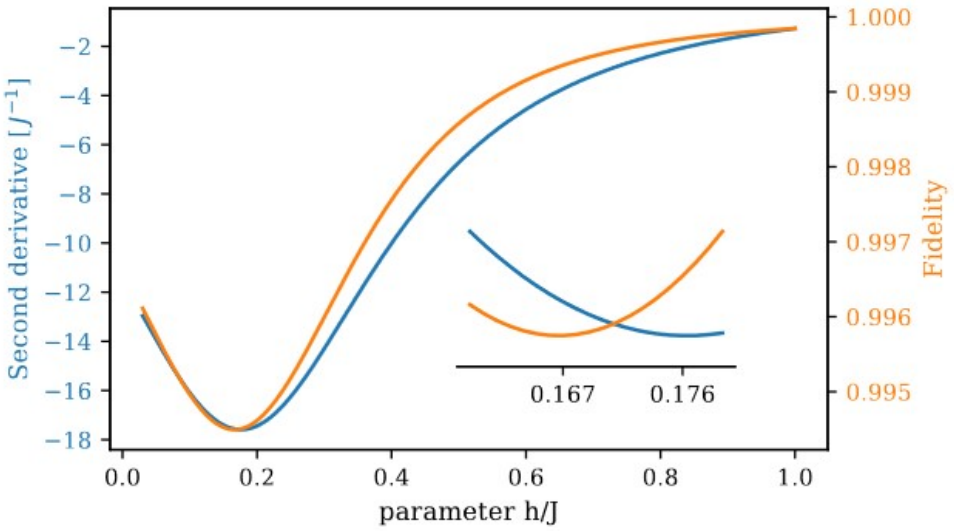


Figure 2.3: Surface code phase transition signatures [17]

The qubits sit on a chain on the quantum chip with two-qubit gate connections indicated by the black lines. The prepared qubit system exhibits 2D characteristics such as a non-zero topological entanglement entropy, a typical feature of 2D systems, although the state preparation only requires 1D connectivity. The Hamiltonian governing the qubit system includes both weight-four and weight-two stabilizers, constructed using Pauli X and Pauli Z operators. The introduction of an external magnetic field in the Z direction introduces a trainsition from the topological phase to the paramagnetic phase. The system Hamiltonian is

$$H = H_{\mathrm{SC}} - h \sum_{i=0}^{n} \sigma_i^z \tag{2.2}$$

where n represents the number of qubits and h the magnetic field strength. The relative strength of the two terms of the Hamiltonian is parameterized by the relative magnetic field strength h, which determines the system's phase. When $h = 0$ ($h = \infty$), the ground state belongs to the topological phase (paramagnetic phase). In the thermodynamic limit, the critical point for the second-order phase transition occurs at $h = 0.3$ [18]. However, in a finite-size system, such as the 9-qubit surface code studied here, the phase transition deviates due to finite-size effects.

In the finite-size system analyzed by [20], we identify the signatures of the phase transition by observing the second derivative of the ground state energy and the ground state fidelity. These quantities exhibit characteristic dips at the critical point, indicating a transition between the topological and paramagnetic phases. In the 9-qubit surface code, the phase transition is observed around $h \sim 0.17$, as shown in Figure 2.3. The dips in Figure 2.3 provide evidence of the ground state crossing the critical point, transitioning from the topological phase to the paramagnetic (trivial) phase.

2.2 Hybrid Neural Network

The following sections outline the architecture of the HNN model and the training procedure, while also offering an intuitive understanding of the cost function in terms of probabilities, providing the necessary mathematical foundation to demonstrate the effectiveness of the training process.

2.2.1 Overview and Circuit Diagram

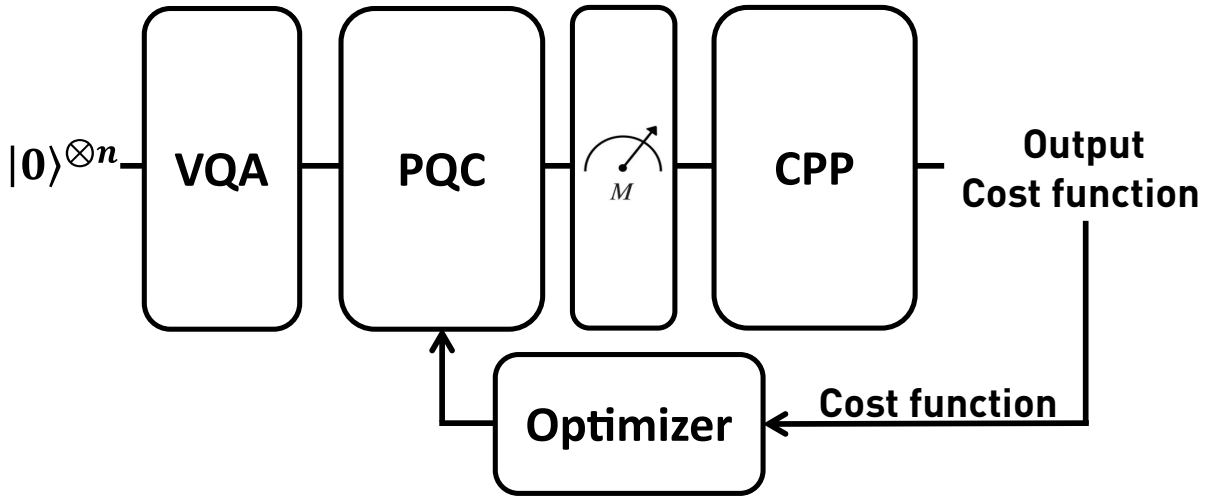


Figure 2.4: HNN diagram, explain the abbreviations in the figure

The structure of the HNN is illustrated in Figure 2.4, which provides an overview of its components. The diagram outlines the process starting from an initial quantum state $|0\rangle^{\otimes n}$, proceeding through a series of quantum and classical units. The components of the HNN include the variational quantum algorithm (VQA), the parameterized quantum circuit (PQC), measurement, and a classical post-processing (CPP) unit. The PQC work under the supervision of a classical optimizer that iteratively updates the gate parameters and minimizes the cost function.

The variational quantum algorithm (VQA) is the first stage in the HNN pipeline. The VQA is a quantum circuit that is optimized in simulations to approximate the ground states of the perturbed Hamiltonian in Eq. 2.2. By optimizing its parameters in the simulations, the output states of the VQA effectively converge to an approximate solution for the desired ground states.

The parameterized quantum circuit (PQC) follows the VQA and serves as the quantum computation core of the HNN. The PQC consists of alternating layers of single-qubit gates and arbitrary-phase controlled-Z (CZ_ϕ) gates. The single-qubit gate applies local Y-rotations to individual qubits, while the CZ_ϕ gate acts on two qubits and implements a continuous set of unitary evolution as in Eq. 2.3.

$$U_{CZ_\phi} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\phi} \end{pmatrix} \quad \text{for } \phi \in [0^\circ, 360^\circ) \tag{2.3}$$

The parameters of the PQC include the rotation angles and phases of these single- and two-qubit gates, which are adjusted during the optimization process to minimize the cost function. This layered structure enables the PQC to efficiently explore the Hilbert space of the quantum system. The CZ_ϕ gate is investigated in detail in [21]. The CZ_ϕ gate operates by dynamically tuning the interaction strength between two qubits through applying flux pulses. The pulses are computed by smoothening a rectangular pulse with a gaussian filter. The pulses modulate the $|g\rangle$-$|e\rangle$ transition frequencies of the two qubits adiabatically, and bring the qubits into resonance at the interaction frequency of $|11\rangle$ and $|20\rangle$. Due to the symmetry in the flux-frequency relation of the transmon qubits, pulse amplitudes that are opposite in sign will result in the same frequency configuration. This fact can be used to suppress long-timescale distortions in the flux signal chain by implementing pulses with an average amplitude of zero (net-zero pulses) [22].

The next stage is the measurement, where correlated single-shot measurements are performed on the quantum state produced by the PQC. The measurement outcomes are processed as length-n bitstrings, where n is the number of qubits in the system. The bitstrings encode information about the quantum state, which is used as the input for subsequent processing.

The classical post-processing (CPP) unit processes the measurement outcomes to produce the output and the final cost function. The CPP unit is parameterized by a vector of parameters $\mathbf{w}$, which are optimized analytically to make the final prediction of the phase of the input state. The combination of PQC and CPP allows the HNN to leverage the strengths of both quantum and classical methods, and the modularization of the classical post-processing facilitates further improvements, such as replacing the CPP by a classical neural network. This is beyond the scope of this work and is proposed as a direction for future research. The optimizer iteratively adjusts the parameters of the PQC to minimize the cost function. By feeding the cost function back into the optimization loop, the system improves its performance until convergence is achieved.

2.2.2 Training set, output and cost function

Training set \ Label	Topological	Non-topological
$\mathcal{D}_{\|0\rangle}$	$\|\psi(h=0)\rangle$	$\|0\rangle^{\otimes n}$ (trivial phase)
$\mathcal{D}_{I/2^n}$	$\|\psi(h=0)\rangle$	$I/2^n$ (virtual in analysis)

Table 2.1: Training sets

A training set is indispensable in supervised machine learning as the training set provides the "experience" for the machine learning model. In supervised learning, the training set consists of a collection of labeled examples, where each input data point is paired with its corresponding output label. By processing the labeled examples, the model learns the underlying structure of the dataset and acquires the ability to generalize to unseen data. To ensure the model's effectiveness in more general applications, the training set is expected to represent as wide range of scenarios the model is likely to encounter as possible. In this experiment, two distinct training sets are used, as in Table 2.1. These training sets are designed to fulfill different objectives of the HNN training. The first training set, denoted as $\mathcal{D}_{I/2^n}$, comprises $|\psi(0)\rangle$ for the topological phase and $|0\rangle^{\otimes n}$ for the non-topological (trivial) phase. This training set is a naive training set as it contains the two typical states from each phase. The second training set, $\mathcal{D}_{|0\rangle^{\otimes n}}$, is designed to distinguish the topological phase states from general trivial quantum states. Specifically, the topological phase state $|\psi(0)\rangle$ is paired with the fully-mixed state $I/2^n$, which is a maximally mixed state used in the analysis to represent trivial quantum states. This second training set provides a more rigorous test of the HNN's capability to classify the topological phase by challenging it to differentiate between the strongly correlated topological phase and a quantum state that lacks such correlations.

The labels in the training set reflect our expectations for the trained HNNs. Specifically, we aim to train the HNNs so that their outputs resemble the probability of an input state belonging to the topological phase. To achieve this, we assign a label of 1 (100% probability) to the topological state and 0 (0% probability) to the non-topological state. The label of the input state is denoted by θ_i in Figure 2.5. The input state is processed by the PQC and measured using single-shot readout in the Z-basis. The single-shot readout result is fed to the classical post-processing unit, which is parameterized by a vector of parameters $\mathbf{w}$. Each measurement outcome is associated with an element in $\mathbf{w}$, in the sense that if the outcome is b, the HNN outputs w_b.

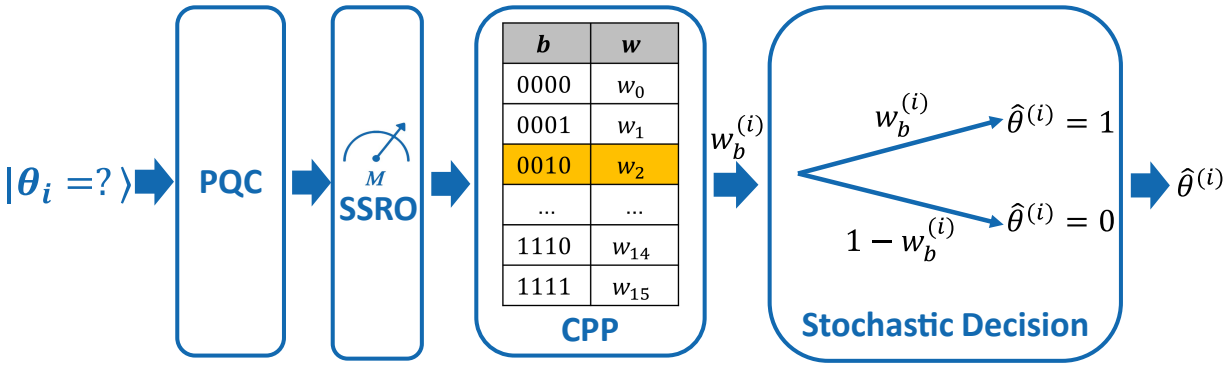


Figure 2.5: HNN diagram with the stochastic decision box. For the simplicity of illustration, we use the 4-qubit system as an example in CPP

To train the HNN, a cost function is needed to assess the correctness of the outputs quantitatively. The cost function should be low when the outputs match the labels of the training

set states and high when they do not. To construct the cost function, we start from the scenario, where the HNN is used to predict the phase of a state from the training set based on the single-shot readout result. Imagine that, we append a stochastic decision box after the HNN (as in Figure 2.5), which receives the output, and makes a stochastic prediction on the phase of the input state: with probability $w_b^{(i)}$ (or $1-w_b^{(i)}$), it decides that the input state belongs to the topological phase (or trivial phase). This decision box is only used for the illustration of the cost function and not implemented in the experiment. The prediction is denoted by $\hat{\theta}^{(i)}$. Given this decision procedure, the probability that the decision box makes a correct prediction is

$$\begin{aligned} p_{cor}^{(i)} &= \begin{cases} w_b^{(i)} & \text{if } \theta_i = 1 \\ 1 - w_b^{(i)} & \text{if } \theta_i = 0 \end{cases} \\ &= \theta_i w_b^{(i)} + (1-\theta_i)(1-w_b^{(i)}) \end{aligned} \tag{2.4}$$

Since the quantum state before measurement is in general a superposition of Z-basis eigenstates, the single-shot readout result follows a distribution over possible outcomes. To suppress the impact of shot noise, we measure each state in the training set for N times, and the decision box is required to predict the phase of the input state based on every single-shot readout result. The probability that the HNN makes no mistakes in the $2 \times N$ predictions is

$$P_{cor} = \prod_{i=1}^{2N} p_{cor}^{(i)} \tag{2.5}$$

There is a factor of 2 before N because the training set contains 2 states. Then we take the logarithm of P_{cor} and divide it by $2N$

$$\begin{aligned} -\frac{1}{2N}\log P_{cor} &= -\frac{1}{2N}\sum_{i=1}^{2N} \log p_{cor}^{(i)} \\ &= -\frac{1}{2N}\sum_{i=1}^{2N} \log(\theta_i w_b^{(i)} + (1-\theta_i)(1-w_b^{(i)})) \\ &\overset{\theta_i\in\{0,1\}}{=} -\frac{1}{2N}\sum_{i=1}^{2N} \theta_i \log(w_b^{(i)}) + (1-\theta_i)\log(1-w_b^{(i)}) \\ &= \mathcal{L}_{BCE} \end{aligned} \tag{2.6}$$

which is the commonly known binary cross entropy loss [23]. It can be further proved that

$$\mathcal{L}_{BCE} = -\frac{1}{2}(\log \mathbf{w} \cdot \mathbf{P_1} + \log(1-\mathbf{w}) \cdot \mathbf{P_0}) \tag{2.7}$$

where $\mathbf{w}$ is the vector of parameters of the classical post-processing unit (and $\log \mathbf{w}$ is the vector of $\log w_b$, so is $\log(1-\mathbf{w})$), P_0, P_1 are the population distributions of the output quantum states over bitstrings (the subscript denotes the label of the input state). The population distribution is further illustrated in Figure 2.6. To suppress the sampling noise, the average output is used to make a stable prediction about the phase of a certain input state. It is simply the average of the HNN outputs for the same input state and computed by taking the dot product of $\mathbf{w}$ and $\mathbf{P}$.

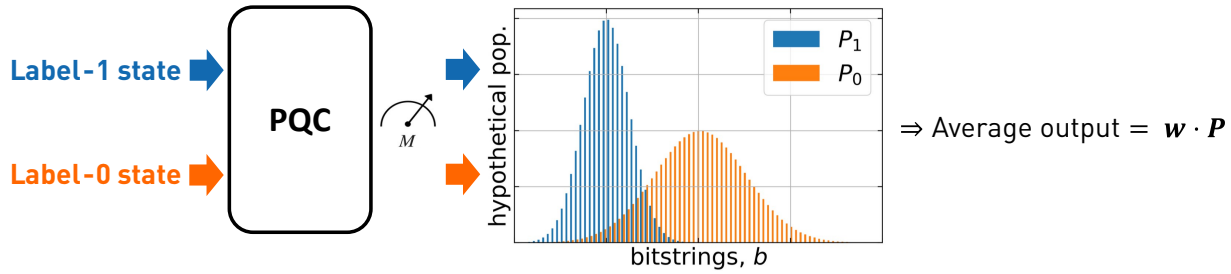


Figure 2.6: HNN diagram with bitstring distributions

2.2.3 Gradient descent optimizer

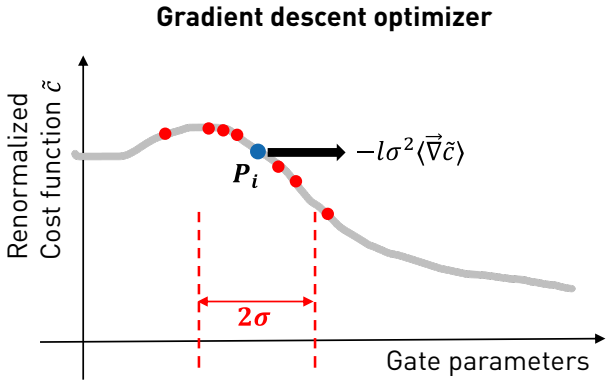


Figure 2.7: Gradient descent algorithm

Algorithm 1 Gradient descent algorithm

Initialize gate parameters at $\mathbf{P_0}$
for $i = 0, 1, 2, \ldots, N$ **do**
  Draw $m-1$ random samples around $\mathbf{P_i}$
  Evaluate cost function c at $\mathbf{P_i}$ (current best guess) and random samples
  Renormalize $\tilde{c} = \frac{c-\bar{c}}{\text{std}(c)}$, estimate gradient $\langle\nabla\tilde{c}\rangle$
  Update $\mathbf{P_{i+1}} = \mathbf{P_i} - l\sigma^2\langle\nabla\tilde{c}\rangle$
end for

The gradient descent algorithm outlined in Algorithm 1 and Figure 2.7 is an iterative optimization procedure designed to update gate parameters to minimize a given cost function. It iteratively evaluates the cost function by randomly sampling around the current parameters, normalizes the cost function to estimate gradients, and updates the parameters against the direction of the gradient. The process is repeated for a specified number of iterations to achieve convergence.

The algorithm begins by initializing the gate parameters at an initial guess $\mathbf{P_0}$. In each iteration i, the algorithm draws $m-1$ random samples around the current best guess $\mathbf{P_i}$ to

explore the local landscape. The sampling process is subject to a gaussian distribution with standard deviation σ. The cost function c is then evaluated at $\mathbf{P_i}$ (the current best guess) as well as at the random samples. To ensure the consistency of the optimizer, the cost function values are renormalized by subtracting the mean and dividing by the standard deviation. The gradient $\nabla\tilde{c}$ of the renormalized cost function is then estimated, and the parameters are updated against the direction of the gradient.

There are several hyper-parameters defined in the algorithm. The first one is the number of samples for estimating the gradient, m. On one hand, the more samples the optimizer draws in a single iteration, the more precise the estimated gradient is. On the other hand, m is limited by the runtime of the experiment. As this optimizer is specially designed for the optimization of a large number of parameters with relatively small number of samples [24], $m = 10$ is used throughout the experiments in this work. The other hyper-parameters are the width of the gaussian distribution $\sigma = 5.4°$ and the learning rate $l = 1.0$.

2.3 Experiment setup

2.3.1 Software structure

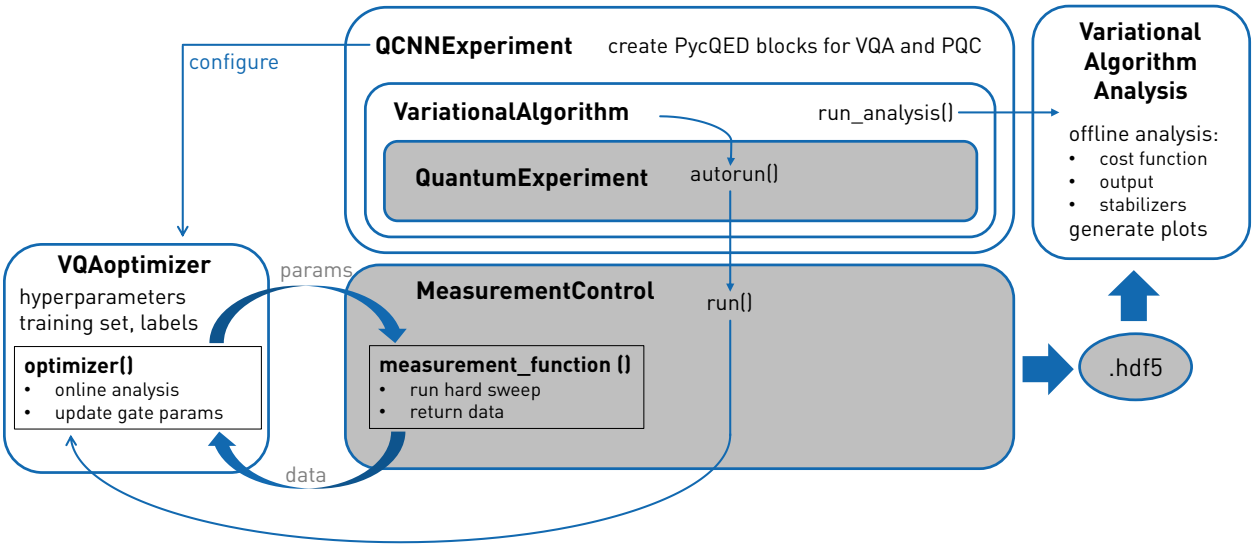


Figure 2.8: software structure

The software structure shown in Figure 2.8 outlines the workflow of the experiment control software. The existing classes are shaded in gray. All the control software is built in the framework of PycQED, the Python-based control software of quantum experiments developed by Qudev. The workflow starts from **QCNNExperiment**, which is a subclass of **VariationalAlgorithm**, which is again a subclass of the existing class **QuantumExperiment**. **QCNNExperiment** prepares the PycQED circuit blocks of VQA and PQC, and calls **VariationalAlgorithm**. The preparation of the blocks is for conducting an adaptive measurement: it is similar to a 2D sweep but the parameters of each hard sweep are

determined iteratively by the optimizer between each call to the hardware. **VariationalAlgorithm** calls *autorun()* of **QuantumExperiment**, and the experiment is passed to **MeasurementControl**. Here, **MeasurementControl** invokes **VQAoptimizer** to take over the optimization process. It handles hyperparameters, training sets, and labels. The *optimizer()* function performs online analysis of the measurement data from **MeasurementControl** and updates the gate parameters accordingly. Measurement data is stored in .hdf5 files, which are later analyzed by **VariationalAlgorithmAnalysis**. This module performs offline analysis of the cost function, outputs, stabilizers, and generates plots for visualization and performance evaluation.

2.3.2 Qubit and gate calibration

Parameter	qb3	qb4	qb5	qb10	qb15	qb14	qb13	qb12	qb7
Lifetime T_1 (µs)	38.5	42.1	54.2	33.3	87.1	51.5	116.3	47.7	114.3
Ramsey decay time, T_2^* (µs)	47.3	37.1	13.4	26.8	31.2	24.7	51.0	40.4	59.1
Echo decay time, T_2^e (µs)	47.7	39.4	16.3	29.4	39.6	32.9	51.9	35.7	101.9
Qubit idle freq., $\omega_Q/2\pi$ (GHz)	3.58	6.23	3.95	6.23	3.88	6.05	3.77	6.10	3.52
Readout freq., $\omega_{RO}/2\pi$ (GHz)	6.93	7.40	6.87	7.09	6.87	7.20	6.98	7.41	6.82
Qutrit readout error, $\epsilon_{RO}^{(3)}$ (%)	4.32	2.78	5.08	4.06	7.90	1.64	2.21	1.63	1.29
Single-qubit RB error, ϵ_{1Q} (%)	0.09	0.12	0.14	0.09	0.13	0.07	0.04	0.08	0.07

Table 2.2: Qubits Performance Data

Qubit and gate calibration results are listed in Table 2.2 and Figure 2.9. The experiment is carried out on a 17-qubit quantum processor as in [25], in the XLD1 experimental setup. The device is fabricated on a high-resistivity silicon substrate with a 150-nm-thick niobium film for superconducting components. The qubits are realized by aluminum-based Josephson junctions, fabricated using electron-beam lithography and shadow evaporation techniques. The qubits are arranged in a diagonal, planar square lattice. It has 9 low-frequency qubits (data qubits) and 8 high-frequency qubits (ancilla qubits) designed for stabilizer experiments. The qubits are capacitively coupled along the edges of the lattice using coplanar waveguides. Single-qubit gates are realized with microwave pulses.

In this work, the qubits are numbered in the same way as in [25]. In total, 9 qubits are used in the experiment as listed in Table 2.2. In addition, we also carried out 4-qubit experiments with qb3, qb4, qb5 and qb10. Preliminary calibrations for approximate single-qubit control and readout had been done by other group members before the HNN experiment. The qubits are initialized in their ground state using a real-time feedback method, which involves qutrit readout followed by corrective single-qubit drive pulses, conditioned on the readout outcome [26, 27].

The calibration result of the CZ_ϕ gates are shown in Figure 2.9. The gates are arranged based on their positions along the qubit chain. Most gate errors are below 3%, except for the

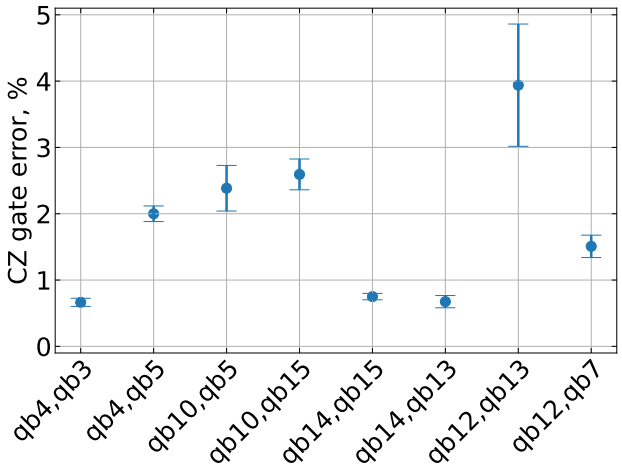


Figure 2.9: Two-qubit gate random benchmarking errors

gate between qb12 and qb13, which has an error rate of approximately 4%. This gate error also exhibits significant variation between calibrations, suggesting the presence of a nearby defect mode.

Chapter 3

Training Experiment on a 4-qubit System

The discussion of the 4-qubit HNN experiment begins with the state preparation measurement, in which stabilizers are measured to ensure that the state preparation circuit and measurement setup are functioning correctly. This step is crucial for ensuring the accuracy of subsequent quantum operations. Then, we use an example of a 4-qubit HNN training experiment to illustrate the principle of the optimizer. The example demonstrates how the HNN is trained with a specific training set, with its cost function decreasing over iterations to convergence. We then introduce two initialization conditions for the following training experiments to investigate the performance of the optimizer in terms of convergence. Then the convergence statistics for the two initialization conditions are studied in detail. Finally, we explore two use cases of the trained HNN to evaluate the performance: in phase boundary detection, the HNN is used to detect phase boundaries by analyzing the set of parameterized ground states; and in single-shot classification, the HNN is employed as a binay classifier to predict the phase of an unknown quantum state based on single-shot readout results. Throughout the discussion, we compare the performance of HNNs trained under different initialization conditions and with different training sets.

3.1 State preparation

A state preparation test experiment is carried out before the training experiment to verify the functionality of the VQA circuit. In the experiment, the ground states of the surface code Hamiltonian perturbed by a magnetic field are prepared by the state preparation circuit (VQA) and measured immediately afterward. The magnetic field is parameterized by h as in Eq. 2.2. The results of the state preparation experiment are illustrated in Figure 3.1, which shows the values of stabilizers for ground states. The x axis of the figure represents the ground states prepared at different values of h, while the y axis shows the measured values of the Z and X stabilizers. On the left side of the plot, all Z and X stabilizers are close to one, which agrees with the fact that $|\psi(0)\rangle$ is the surface code ground state, stabilized by these.

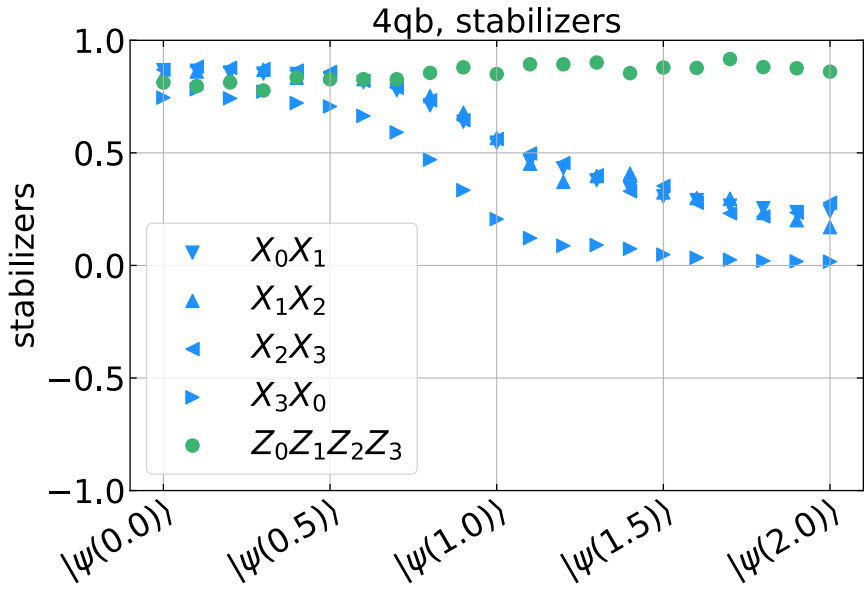


Figure 3.1: 4-qubit system stabilizer measurement

In the remaining part of the plot, the measured Z stabilizer remains close to 1 across all ground states, as shown by the green dots in the figure. This behavior is expected because the Z stabilizer commutes with the magnetic field in Z direction. As a result, its values are unaffected by the perturbation introduced by the field. In contrast, the measured X stabilizers (represented by blue triangles) exhibit a noticeable decrease as the parameter h increases. Starting near 1, the values of the X stabilizers drop toward 0 as the ground state exits the topological phase and transitions into the paramagnetic phase. This phenomenon reflects the sensitivity of the X stabilizers to the perturbing magnetic field, as they do not commute with the Z-field. These experiment results validate the correctness of the state preparation and measurement procedure.

3.2 Training experiment

Figure 3.2 illustrates the results of a training experiment conducted for a 4-qubit HNN. The plot shows the cost function against training iterations with the $\mathcal{D}_{I/2^n}$ training set. The circuit parameters were initialized close to a known global optimum obtained from simulations (the optimal circuit is the mirror image of the preparation circuit of $|\psi(0)\rangle$). As observed, the cost function decreases steadily over the training iterations and eventually converges to a lower value, indicating successful optimization.

The blue and red dots in the plot represent the samples drawn during each training iteration. Specifically, the blue dots at the bottom of the traces during the final phase of the training correspond to the current best guess points. This phenomenon indicates that the optimizer effectively converges to at least a local optimum, as the random samples around the current best guess consistently have higher cost function values. Table 3.1 summarizes key tim-

ing parameters of the experiment, including acquisition shots, the number of sequences and segments, and the runtime based on the repetition rate only (approximately 54 seconds). Sequences and segments are the fundamental units of the quantum experiment implemented by PycQED. The real runtime is about 12 minutes, and the corresponding overhead is mostly due to the time spent on configuring the AWGs.

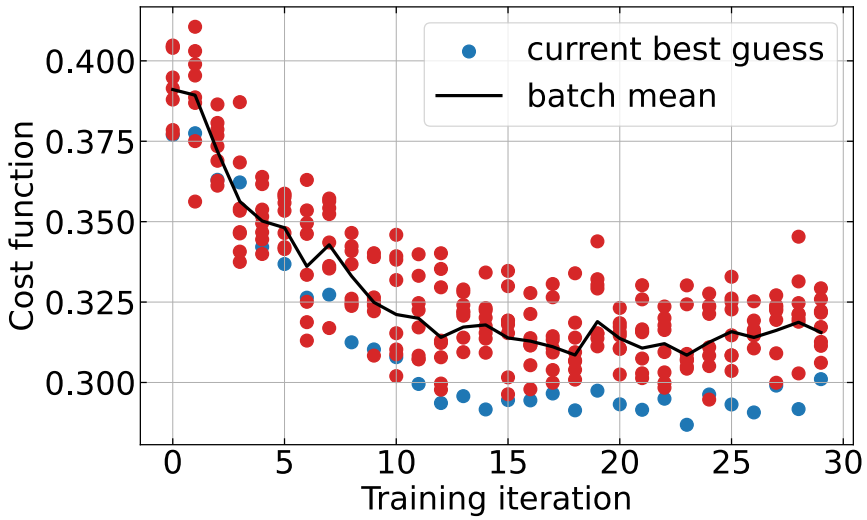


Figure 3.2: 4-qubit HNN training cost function

Parameter	Value
Acquisition shots	2^{13}
$n_{seq} \times n_{seg}$	30×10
Repetition period	22μs
Total	54s

Table 3.1: Timing parameters

To further validate the training results, Figure 3.3 shows the gate parameters before and after the training experiment. The correspondence between parameters and quantum gates are depicted in Figure 3.4. Parameters of consecutive RY gates are summed up to simplify the plot. The trained parameters are compared against the optimal parameter values obtained from simulations. It can be seen that the optimizer successfully drives the parameters toward the known global optimum, confirming the convergence observed in Figure 3.2. However, a slight deviation between the trained parameter values and the optimal values is observed. This deviation can be attributed to the flatness of the cost function landscape near the global optimum. Notably, subsequent experiments demonstrated that the trained circuit performs comparably to the theoretical global optimal circuit, suggesting that the trained circuit parameters are robust and effective despite the observed deviations.

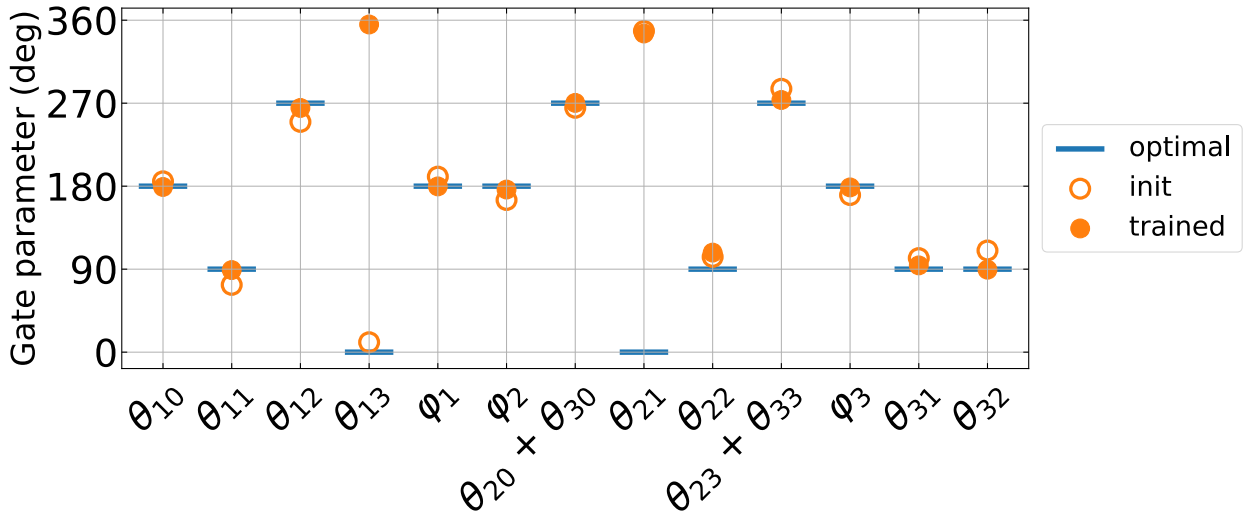


Figure 3.3: Trained parameter values of the 4-qubit HNN

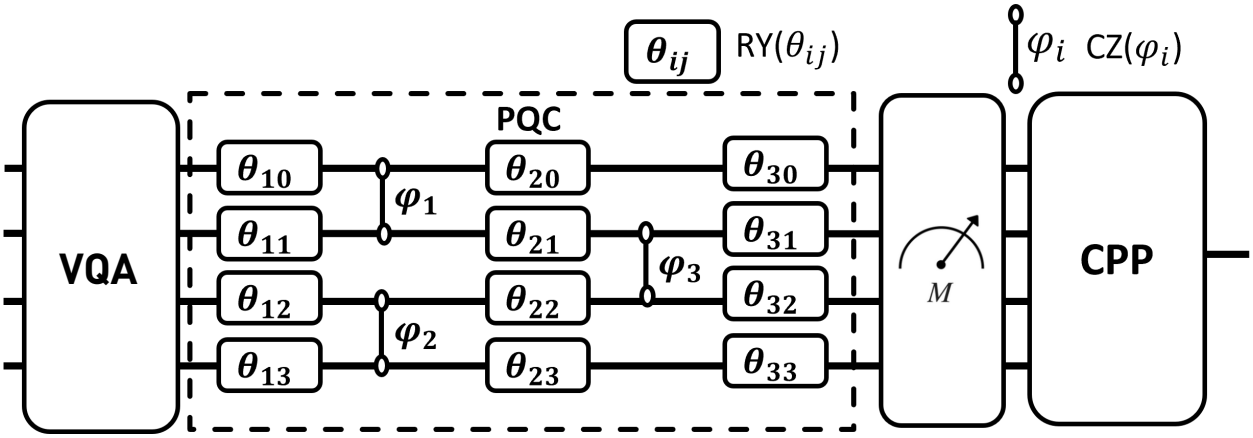


Figure 3.4: Circuit diagram of the 4-qubit HNN

3.3 Concept of the verification experiments

To fully evaluate the performance of the trained HNN , it is crucial to test them on practical use cases that reflect their intended applications. While training metrics like cost function convergence provide insights into optimization, they do not directly measure the HNN's ability to solve real-world problems. Use cases, such as phase boundary detection and single-shot classification, allow for a more comprehensive evaluation by testing the trained HNNs on diverse datasets and tasks.

3.3.1 Use case: phase boundary detection

In this use case, the trained HNN is employed to detect the phase transition boundary as illustrated in Figure 3.5. In this experiment, we extract the trained parameter values in the training experiment that generate the lowest cost function, and reuse them in the PQC to

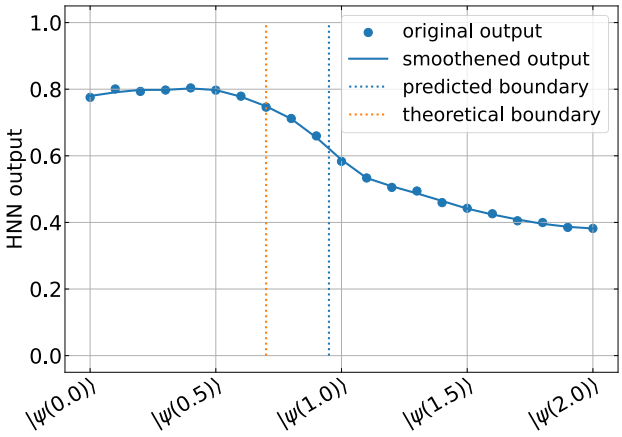


Figure 3.5: 4qb phase boundary prediction, trained with fms, expert init

generate outputs for the continuously parameterized ground states. For each ground state, the ouputs are averaged as in Figure 2.6 to produce the data points shown as blues dots in Figure 3.5. These outputs represent the HNN's predictions of the likelihood that the ground state being a topological phase state. To accurately determine the phase transition boundary, the averaged outputs were further processed using a Savitzky-Golay filter to smoothen the data. The filter's window length is 5 and the order of the polynomial used to fit the samples is 2. The smoothened output is represented by the blue line in the plot. The derivative of this smoothened output was then calculated, and the position corresponding to the maximum derivative was identified as the predicted phase transition boundary. This boundary is indicated by the blue dashed line in the plot. For comparison, the theoretical phase boundary is marked by the orange dashed line. The following data analysis section presents a comprehensive evaluation of the trained HNN's phase boundary prediction capabilities across various training examples to illustrate the training effectiveness.

3.3.2 Use case: single-shot classification

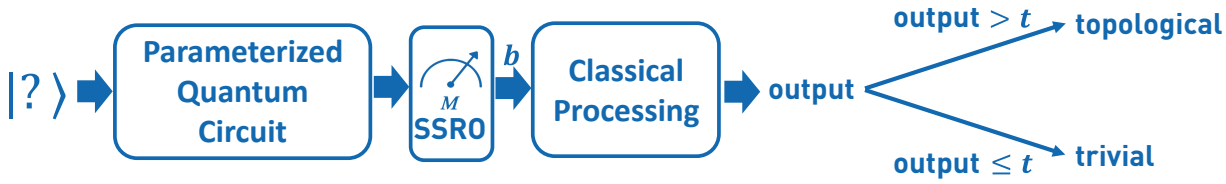


Figure 3.6: HNN as a binary classifier

The trained HNN can also be utilized as a parameterized binary classifier to distinguish between topological and trivial phases, as illustrated in Figure 3.6. In this use case, a quantum state taken from an unknown phase is fed into the parameterized quantum circuit of the trained HNN. The output quantum state is then measured once using single-shot readout (SSRO), producing a bitstring b. This bitstring is subsequently processed by the classical post-processing unit, which generates a single output value. The phase of the input quantum

state is predicted by applying a threshold t to this output: if the output exceeds t, the input state is classified as belonging to a topological phase; otherwise, it is classified as a trivial phase state.

Ground truth \ Prediction	**Positive**	**Negative**
Positive	True Positive (TP)	False Positive (FP)
Negative	False Negative (FN)	True Negative (TN)

Table 3.2: Categorized performance of a binery classifier

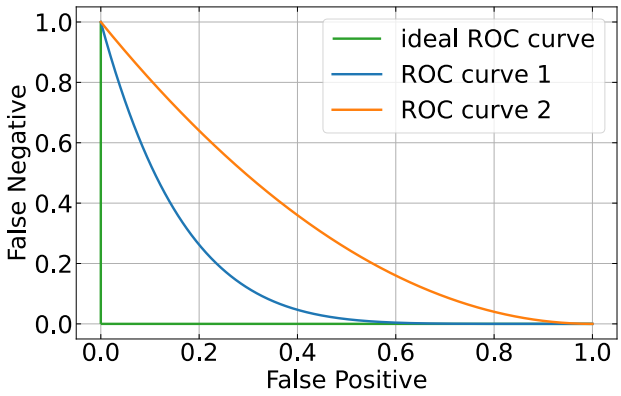


Figure 3.7: Concept of ROC curves. See the text for details

The receiver operating characteristic (ROC) curve is used to evaluate the performance of the trained HNN as a parameterized binary classifier. The ROC curve is a standard tool for assessing the accuracy of binary classifiers [28]. The performance of the binary classifier is categorized into four dimensions, as illustrated in Table 3.2: true positive (TP), false positive (FP), false negative (FN), and true negative (TN). Each of these scenarios is associated with a probability of occurrence. For this analysis, the focus is on the two error probabilities: the probability of FP and the probability of FN. To provide a comprehensive evaluation of the classifier's performance across different parameter values, we plot the probability of FP against the probability of FN to get the ROC curve.

Figure 3.7 illustrates a conceptual representation of ROC curves for different binary classifiers. The ideal binary classifier is represented by the green line in Figure 3.7. In this case, with the correct parameter, the probabilities of both FP and FN are zero, indicating perfect classification. But, in practical scenarios, such as when a classifier is tasked with distinguishing between two overlapping probability distributions based on observed samples, perfect separation is impossible. Realistic binary classifiers have ROC curves resembling the blue and orange lines. Among these, the blue curve is considered superior to the orange curve as it is closer to the ideal ROC curve. This proximity signifies that the blue classifier achieves higher accuracy by making fewer errors in prediction tasks, thus demonstrating better overall performance.

3.4 Experiment data analysis

After outlining the principles behind the training and verification experiments, we proceed to analyze the experimental data and extract useful information from the analysis. The convergence statistics under different initialization conditions are analyzed first in the following section.

3.4.1 Comparison of initialization conditions

Expert and random initialization

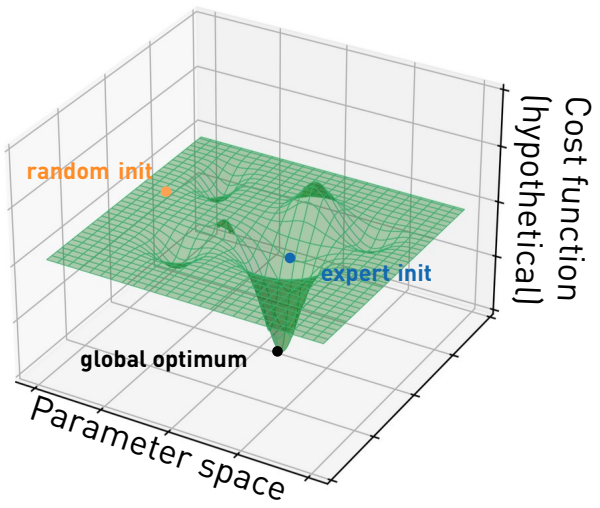


Figure 3.8: Hypothetical cost function landscape, sketched to illustrate the two different initializations

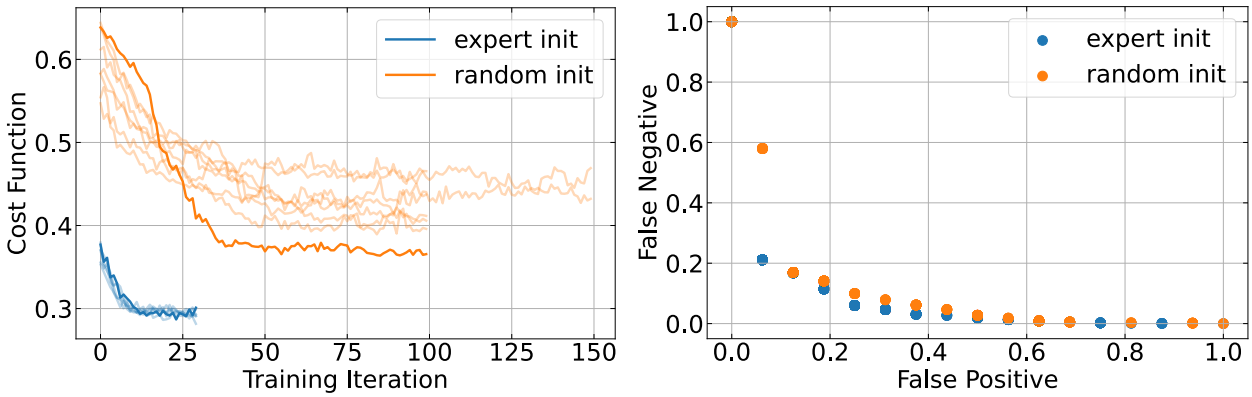
Figure 3.8 presents a hypothetical schematic of the cost function landscape, used to illustrate two different initialization conditions. This schematic is purely illustrative and does not originate from experimental data or simulations. The global optimum of the cost function, corresponding to the known global optimal circuit for the $\mathcal{D}_{I/2^n}$ training set, is represented by the black dot at the bottom of the cost function landscape.

In the training experiments, two distinct initialization conditions are employed. We call the first one expert initialization, where the quantum gate parameters are initialized close to the known global optimum. These parameters are drawn from a uniform distribution within a small range around the optimal values, specifically $\theta^{\otimes n} \sim \mathcal{U}(\theta_{opt} - 22.5°, \theta_{opt} + 22.5°)^{\otimes n}$. This ensures that the initial parameters are near the global minimum of the cost function, making achieving convergence to the global optimum easier for the optimizer. The second condition is random initialization, where the circuit parameters are initialized randomly across the entire parameter space. In this case, the parameters are sampled from a uniform distribution over the full range of possible values, $\theta^{\otimes n} \sim \mathcal{U}(0°, 360°)^{\otimes n}$. This approach assumes no

prior knowledge of the global optimum, simulating a less informed starting point for the optimization process.

These two initialization strategies are designed to assess the impact of prior knowledge on the training process and the ability of the optimizer to converge toward the global optimum under different starting conditions. Furthermore, it provides a comparison of the training results with the known global optimum

Cost function traces under different initialization conditions



(a) Cost function traces **(b)** ROC curves

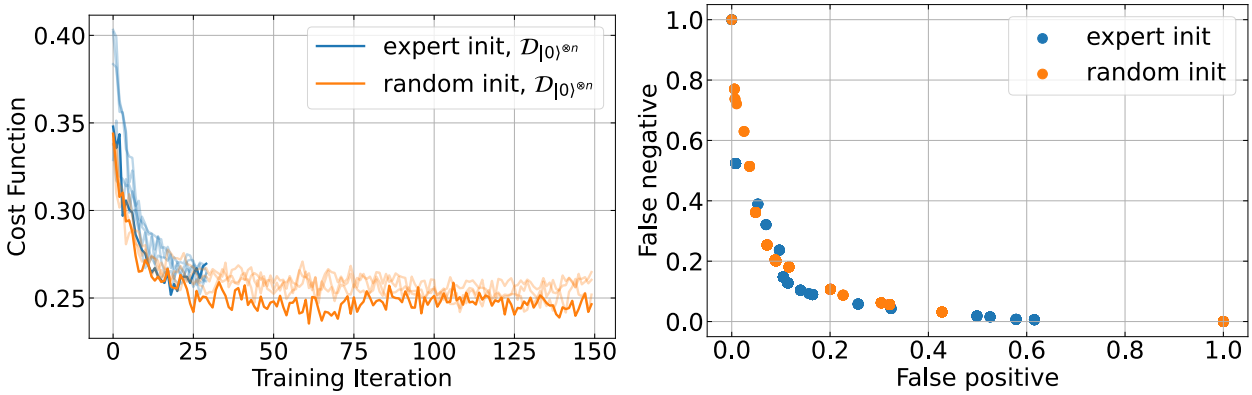
Figure 3.9: Convergence statistics of the 4-qubit HNN training experiments with the $\mathcal{D}_{I/2^n}$ training set. The ROC curve is parameterized by the threshold t as in Figure 3.6. The two ROC curves correspond to the solid cost function traces on the left.

Figure 3.9 presents the results of training experiments under different initialization conditions. Figure 3.9a shows multiple instances of training experiments, with each trace representing the cost function's evolution during training. The blue traces correspond to experiments with expert initialization, while the orange traces represent experiments with random initialization.

The results indicate that experiments with expert initialization converge significantly faster. Moreover, a comparison of the trained gate parameter values reveals that all experiments with expert initialization converge near the known global optimum. In contrast, training with random initialization begins at higher cost function values and, although it still converges, none of the instances reach the cost function values near the global optimum. This suggests that with random initialization, the optimizer is more likely to become trapped in a local optimum.

To evaluate the performance of the trained HNNs under these different initialization conditions, the trained HNN with the lowest cost function from each condition was selected, and their ROC curves were plotted in Figure 3.9b. The ROC curves demonstrate that the HNN trained with expert initialization slightly outperforms the one trained with random

initialization but the difference is only marginal. This result shows that reaching the global optimum during training is not strictly necessary. As long as the solution found by the optimizer performs well in the use-case test, the training experiment can be deemed successful. This is similar to the mindset of classical machine learning.



(a) Cost function traces **(b)** ROC curves

Figure 3.10: Convergence statistics of the 4-qubit HNN training experiments with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set

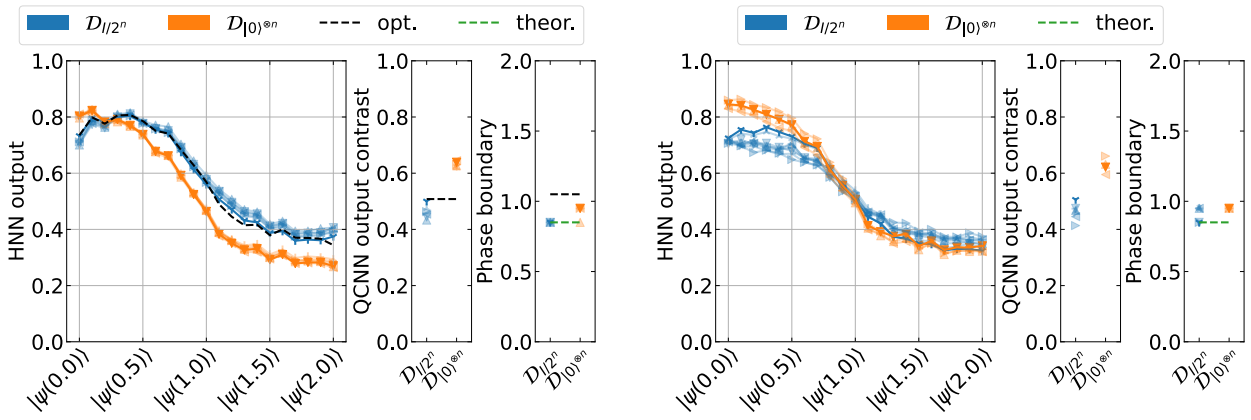
Figure 3.10 presents the results of training experiments using the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set under both expert and random initialization conditions. Unlike the results observed with the $\mathcal{D}_{I/2^n}$ training set (Figure 3.9), there is no significant difference between the expert and random initialization conditions in terms of cost function values or ROC curves. The exact reason for the easy convergence is still under discussion due to a lack of direct experimental evidence, because it is impractical to explore the cost function landscape, which exists as a manifold in a high-dimensional space. A hypothetical explanation for this observation is the existence of multiple optimal circuits when using the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set. This abundance of optimal solutions allows the optimizer to find an effective circuit even when starting from random initialization. In contrast, the $\mathcal{D}_{I/2^n}$ training set presents a more constrained optimization landscape, where expert initialization provides a distinct advantage by guiding the optimizer toward the global optimum. An alternative explanation is that the cost function landscape is more convex, which makes the convergence easier for the optimizer.

3.4.2 Comparison of training sets

Besides the initialization conditions, the following sections dedicate to compare the two training sets in terms of phase boundary detection and single-shot classification.

Phase boundary prediction

Figure 3.11 compares the training results of different training sets in the context of phase transition boundary detection. Figure 3.11a corresponds to experiments with expert initialization, and Figure 3.11b shows the results for random initialization. In each figure, the average HNN outputs for the continuously parameterized ground state set are depicted in



(a) Expert init **(b)** Random init

Figure 3.11: 4-qubit HNN phase boundary detection results. Each plot contains 3 panels. The left panel shows the HNN output against the set of parameterized ground states. The middle and right panels show the contrast of output and the predicted phase boundary, respectively.

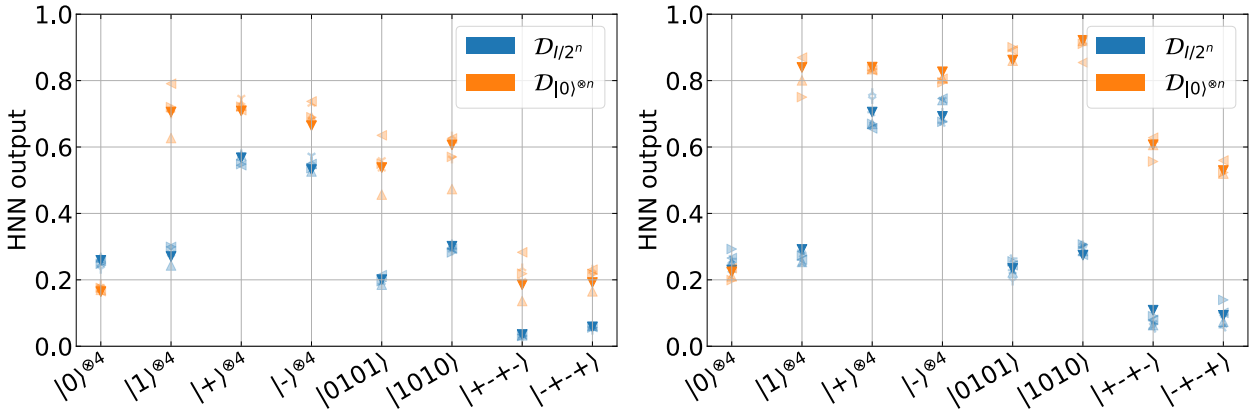
the left, which are used to determine the phase boundary. The middle panel illustrates the contrast in the average HNN outputs, while the right shows the predicted phase boundaries compared to the theoretical values. The black dashed lines represent the data of the global optimal circuit. The training sets are encoded by color: blue represents the $\mathcal{D}_{I/2^n}$ training set, while orange corresponds to the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set.

HNNs trained with the $\mathcal{D}_{I/2^n}$ training set produce slightly more precise phase boundary predictions, as the predicted phase boundary is closer to the theoretical value in the third panel of Figure 3.11a. However, due to the limited number of sample points (only 21 states between $h = 0$ and $h = 2$), the overall precision of the phase boundary predictions is inherently constrained. With the absence of training data near the phase boundary, the detection is not expected to be precise. Therefore, it cannot be definitively concluded that the $\mathcal{D}_{I/2^n}$ training set is superior to the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set for phase boundary detection.

The black dashed lines in Figure 3.11a represent the outputs of the global optimum circuit (optimized in theory and evaluated in hardware), which are nearly identical to those of the trained circuits and do not result in any significant improvement in phase boundary detection. This observation indicates that the cost function landscape is relatively flat near the global optimum, resulting in similar performance for all circuits within this region. Figure 3.11b shows the results for HNNs trained with random initialization. Similar conclusions can be drawn, indicating that the choice of initialization does not significantly impact the HNN's performance in phase boundary detection for the given training sets.

Average outputs on the trivial states

Thus far, the performance of the trained HNNs has been evaluated only on continuously parameterized ground states and the virtual fully-mixed state. However, in real-world appli-



(a) Expert init **(b)** Random init

Figure 3.12: Average outputs of HNNs on trivial quantum states. The HNN trained with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set tends to generate higher outputs for the trivial quantum states.

cations, there exists a much broader range of quantum states. To explore the performance of the trained HNNs on a more diverse set of quantum states, Figure 3.12 presents their average outputs when tested on a set of trivial quantum states. These states are simple product states belonging to trivial quantum phases. Similar to the phase boundary detection results, Figure 3.12a shows the outputs of HNNs trained with expert initialization, while Figure 3.12b displays the results for HNNs trained with random initialization. The training sets are color-coded as before: blue represents the $\mathcal{D}_{I/2^n}$ training set, and orange corresponds to the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set.

For both initialization conditions, HNNs trained with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set exhibit higher outputs on the trivial states. This outcome is attributed to the fact that these HNNs converge to circuits that only recognize the $|0\rangle^{\otimes n}$ state. For example, the identity circuit is one of the optimal circuits for distinguishing the $|0\rangle^{\otimes n}$ state from topological states, according to simulations. However, such a circuit does not fulfill the broader objective of distinguishing general topological states from trivial states, because the classification task with the identity circuit as the PQC is the same as direct measurement, which is proved inefficient in literature for the purpose of recognizing topological states.

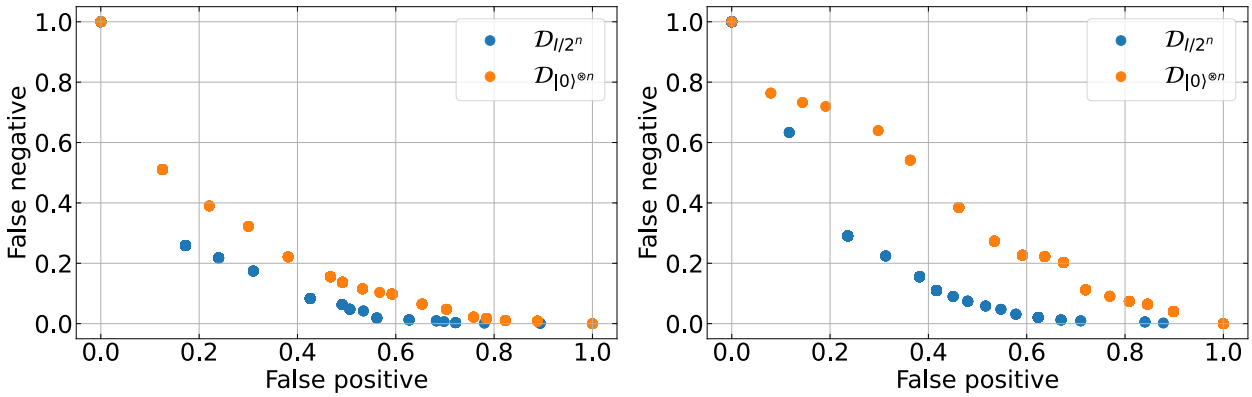
As discussed in the previous sections, HNNs trained with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set performed better on continuously parameterized ground states, while those trained with the $\mathcal{D}_{I/2^n}$ training set were more effective in distinguishing topological states from trivial states. To provide a comprehensive evaluation of the trained HNNs, it is essential to test them on a verification set comprising states from both the parameterized ground states and the trivial states. This would allow for a thorough comparison of the HNNs' overall performance and their ability to generalize across a diverse range of quantum states.

ROC curve

The more comprehensive verification set is shown in Table 3.3. It comprises 4 topological states from the continuously parameterized ground states and 8 states from the trivial quantum states. We will use ROC curves to evaluate the trained HNNs' ability to make correct predictions on the comprehensive verification set. Although there are less topological states than trivial states, their contribution to the false positive and false negative probabilities are balanced in data analysis.

Topo.	$\|\psi(0.0)\rangle$	$\|\psi(0.1)\rangle$	$\|\psi(0.2)\rangle$	$\|\psi(0.3)\rangle$	-	-	-	-
Trivial	$\|0\rangle^{\otimes 4}$	$\|1\rangle^{\otimes 4}$	$\|+\rangle^{\otimes 4}$	$\|-\rangle^{\otimes 4}$	$\|0101\rangle$	$\|1010\rangle$	$\|+-+-\rangle$	$\|-+-+\rangle$

Table 3.3: The comprehensive verification set



(a) Expert init **(b)** Random init

Figure 3.13: ROC curve computed based on the comprehensive verification set

Figure 3.13 shows the ROC curves for the trained HNNs tested on the comprehensive verification set, with the same color codes. Figure 3.13a represents results for HNNs trained with expert initialization, while Figure 3.13b corresponds to HNNs trained with random initialization.

In Figure 3.13a, the difference between the results of the two training sets is smaller compared to Figure 3.13b. This can be attributed to the smaller initialization range in expert initialization, which causes the circuits to converge to similar gate parameters. This leads to less variation in performance, regardless of the training set. Conversely, with random initialization (Figure 3.13b), a wider initialization range allows the circuits to converge to more diverse gate parameters, resulting in a more noticeable difference in performance between the two training sets.

It can be concluded that, on the comprehensive verification set, the $\mathcal{D}_{I/2^n}$ training set enables the trained HNNs to better distinguish topological phase states from all other states. This suggests that the $\mathcal{D}_{I/2^n}$ training set is more effective than the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set for a

broader range of applications.

This concludes the discussion of the 4-qubit HNN training and verification experiments. In the next chapter, we will scale up to a 9-qubit system and investigate the performance of 9-qubit HNNs on similar tasks.

Chapter 4

Comparison with a 9-qubit Experiment

In this chapter, we extend the scope of our investigation to a 9-qubit system. Based on the principles and conclusions gained from the 4-qubit HNN experiments, we aim to explore the scalability of the HNN framework to larger quantum systems. This chapter discusses the similar topics, but with a focus on the difference induced by the increased complexity of the system.

4.1 State preparation

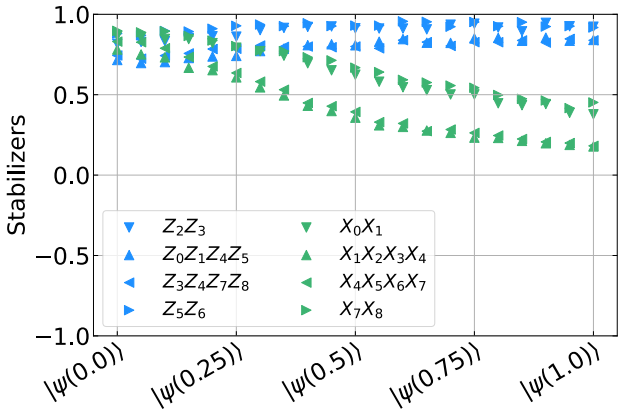


Figure 4.1: 9qb stabilizers

Figure 4.1 displays the stabilizer measurement results for the 9-qubit state preparation experiment. The stabilizers are measured across the parameter range $h \in [0, 1]$. For $|\psi(0)\rangle$, the stabilizers consistently approach the ideal value 1, confirming that the state preparation and measurement process work correctly. As the parameter h increases, the Z stabilizers remain close to 1 while the X stabilizers decrease to near 0 because of their different commutation relations with the Z field perturbation. The edge stabilizers have larger values because they are less prone to single-qubit errors. Compared to the 4-qubit stabilizer measurements,

the 9-qubit results exhibit similar trends and are consistent with the previous observations. However, the 9-qubit system shows more complexity in the stabilizer patterns due to the increased number of qubits and stabilizer operators.

4.2 Experiment data analysis

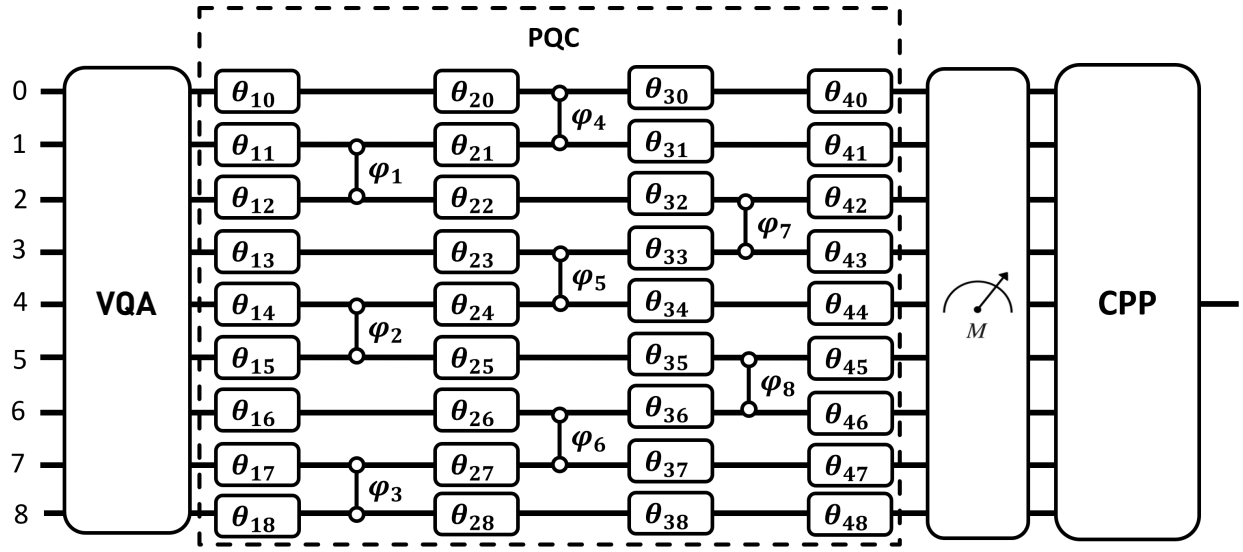
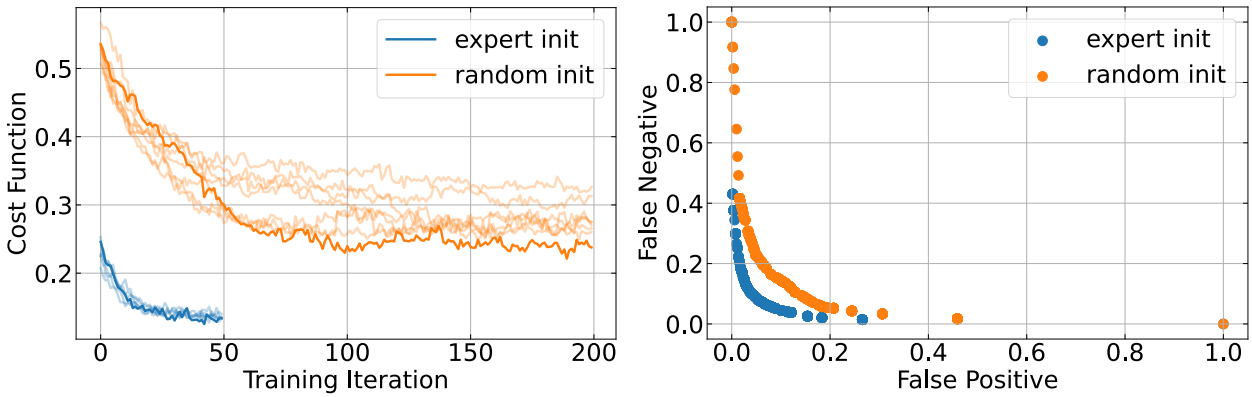


Figure 4.2: Circuit diagram of the 9-qubit HNN

After confirming the functionality of the state preparation process through stabilizer measurements, we proceed to analyze the 9-qubit HNN training experiments. The 9-qubit system exhibits considerable increase of complexity, as visible in the circuit diagram in Figure 4.2. The circuit follows the structure of alternating single-qubit gates and CZ_ϕ gates, but with more layers compared to the 4-qubit circuit to better process the intricacy of 9-qubit states. This is because the PQC is designed to include the mirror image of the state preparation circuit (VQA) as it is the optimal circuit from the simulation. By leveraging the insights gained from the 4-qubit experiments, we aim to understand how the increased system size influences the training dynamics and performance of the HNN.

4.2.1 Comparison of initialization conditions

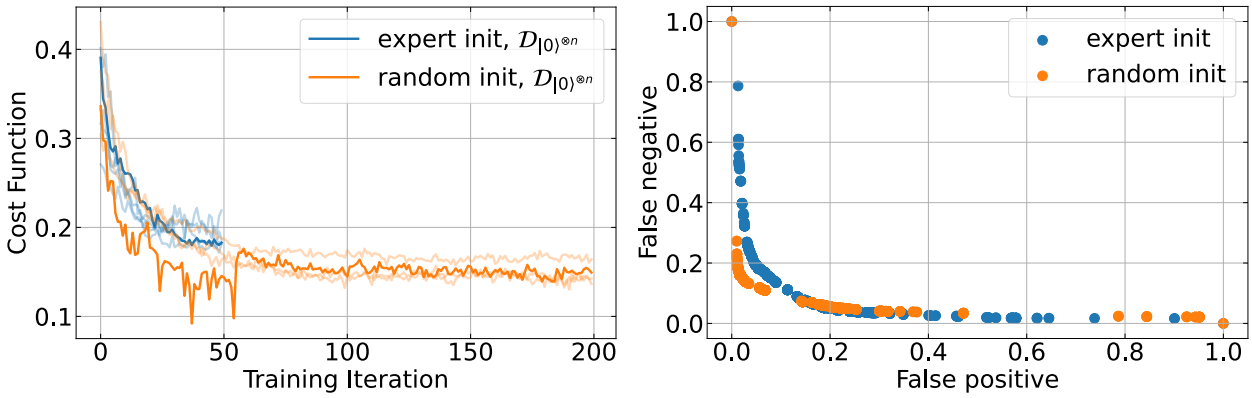
Figure 4.3 shows the convergence statistics of the 9-qubit HNN training experiments under expert (blue) and random (orange) initialization conditions, with training results evaluated using ROC curves. Figure 4.3a illustrates the cost function over training iterations, where expert initialization leads to faster and more consistent convergence to low cost function values. In contrast, random initialization results in a broader range of convergence behaviors, with all traces converging to higher cost function values, indicating that the optimizer is trapped in local optima, as in the 4-qubit experiment.



(a) Cost function traces **(b)** ROC curves

Figure 4.3: Comparison of convergence under different initialization conditions with the $\mathcal{D}_{I/2^n}$ training set. ROC curves are evaluated with the training set. The ROC curves in (b) correspond to the solid cost function traces in (a).

The ROC curves in Figure 4.3b reveal a larger difference between the performance of the two initialization conditions compared to the 4-qubit case. HNNs trained with expert initialization show marginally better classification performance, with the ROC curve closer to the ideal one. We speculate that the added complexity of the 9-qubit system makes it more difficult for the optimizer to find a circuit that is similar to the optimal circuit, and amplifies the difference in performance.



(a) Cost function traces **(b)** ROC curves

Figure 4.4: Comparison of convergence under different initialization conditions with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set. ROC curves are evaluated with the training set.

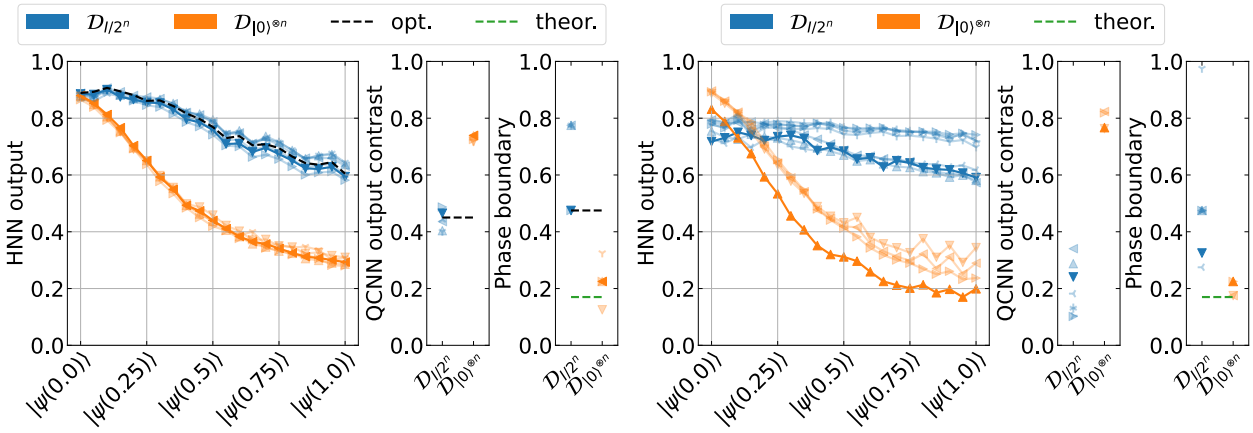
In Figure 4.4, the convergence behavior is consistent with the results observed in the 4-qubit experiments and can be explained using similar reasoning. The cost function converges to comparable values regardless of the initialization condition, indicating that both expert and random initialization result in circuits that perform similarly. Furthermore, the ROC curves show no significant difference between the two initialization conditions, suggesting that the increased system size does not amplify variability in this case.

An anomaly is observed during one of the training experiments, where the cost function initially drops to a value even lower than the global optimum before returning to normal levels. This behavior is likely caused by random, transient noise, as the trained HNN with this unusually low cost function does not outperform other training results. Furthermore, the cost function stabilizes to normal values in the later stages of training, indicating that the anomaly is not due to a systematic error.

4.2.2 Comparison of training sets

As in the previous section, we continue the comparison of training sets in a broader context. The trained HNNs are first used to predict the phase boundary in the parameterized ground states, and then tested on a more comprehensive verification set.

Phase boundary prediction



(a) Expert init **(b)** Random init

Figure 4.5: 9-qubit HNN phase boundary detection results. Each plot contains 3 panels. The left panel shows the HNN output against the set of parameterized ground states. The middle and right panels show the contrast of output and the predicted phase boundary, respectively.

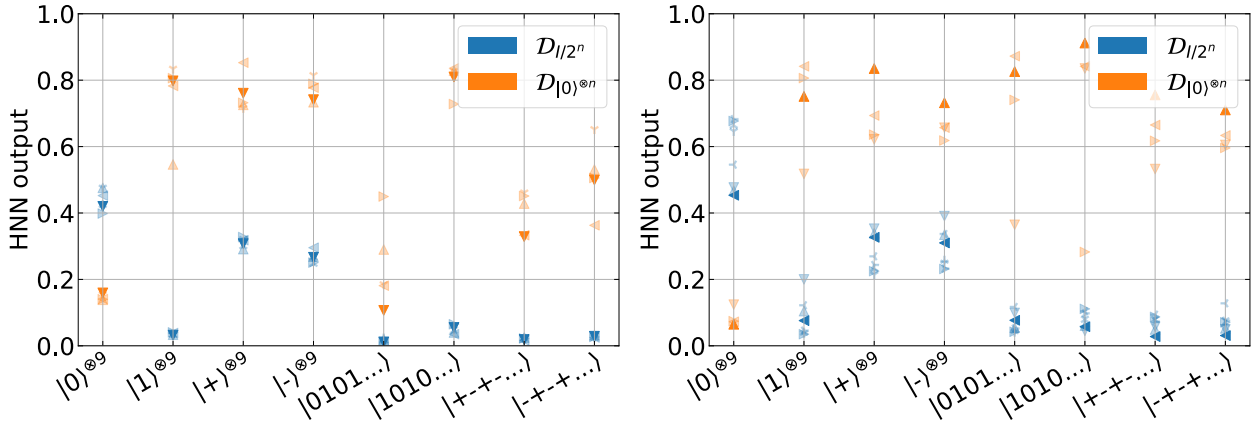
The average outputs of the HNNs trained under random initialization exhibit greater variability compared to the 4-qubit results, as in Figure 4.5b. This increased variability is likely due to the larger parameter space in the 9-qubit system, which allows the optimizer to converge to a wider range of circuit configurations. This behavior highlights the challenges introduced by the increased system size, as the random initialization leads to more diverse solutions, some of which may deviate significantly from optimal performance.

Compared with the 4-qubit experiments, the output contrast for the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set in the 9-qubit case is significantly larger than that observed for the $\mathcal{D}_{I/2^n}$ training set. Moreover, HNNs trained with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set produce more precise phase boundary predictions

under both initialization conditions. This improvement suggests that, in the 9-qubit system, the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set leads to circuits that are better suited for phase boundary detection. But, as stated in the 4-qubit chapter, this does not suffice to assert that the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set is superior to the $\mathcal{D}_{I/2^n}$ training set without further investigation.

Finally, as observed in the 4-qubit experiments, the theoretical optimal circuit does not outperform the training results of the HNNs as displayed in Figure 4.5a. This finding supports the conclusion that the cost function landscape is relatively flat near the global optimum, leading to similar performance for circuits in this region. The flatness of the landscape ensures that a variety of trained circuits, even those not perfectly optimized, can achieve comparable levels of performance in phase recognition tasks.

Average outputs on the trivial states



(a) Expert init **(b)** Random init

Figure 4.6: Average outputs on the trivial quantum states. The HNN trained with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set tends to generate higher outputs for the trivial quantum states.

The average HNN outputs on the trivial quantum states are displayed in Figure 4.6. HNNs trained with the $\mathcal{D}_{I/2^n}$ training set provide more accurate phase predictions for all trivial states except the $|0\rangle^{\otimes n}$ state, consistent with the observations from the 4-qubit results. Notably, the difference in average outputs between the $\mathcal{D}_{I/2^n}$ and $\mathcal{D}_{|0\rangle^{\otimes n}}$ training sets is more significant in the 9-qubit system compared to the 4-qubit case. Since the output is the predicted probability that a given input state belongs to the topological phase, it can be reasonably inferred that the ROC curves for the two training sets will show greater separation in the subsequent ROC curve comparisons on the comprehensive verification set.

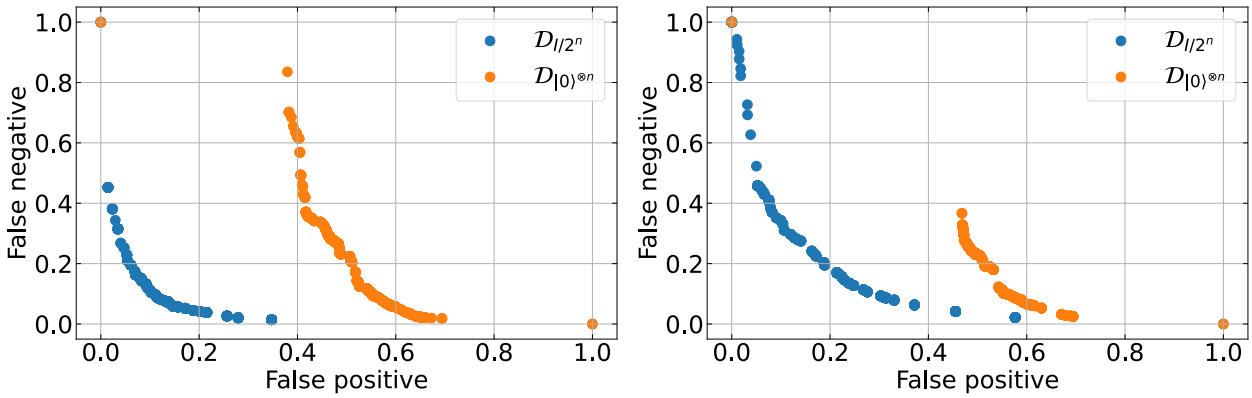
ROC curve

Table 4.1 shows the comprehensive training set for the 9-qubit HNN verification experiment. The data is analyzed in the same way as the 4-qubit case, such that the topological and

Topo.	$\|\psi(0.0)\rangle$	$\|\psi(0.05)\rangle$	$\|\psi(0.1)\rangle$	$\|\psi(0.15)\rangle$	-	-	-	-
Trivial	$\|0\rangle^{\otimes 9}$	$\|1\rangle^{\otimes 9}$	$\|+\rangle^{\otimes 9}$	$\|-\rangle^{\otimes 9}$	$\|0101...\rangle$	$\|1010...\rangle$	$\|+-+-...\rangle$	$\|-+-+...\rangle$

Table 4.1: Comprehensive verification set

trivial states contribute equally to the error probabilities.



(a) Expert init **(b)** Random init

Figure 4.7: ROC curves computed based on the comprehensive verification set

Figure 4.7 presents the ROC curves for the 9-qubit HNNs verified using the comprehensive verification set. The ROC curves are shown for HNNs trained with the $\mathcal{D}_{I/2^n}$ training set and the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set under both expert initialization (Figure 4.7a) and random initialization (Figure 4.7b). HNNs trained with the $\mathcal{D}_{I/2^n}$ training set demonstrate lower error probabilities compared to those trained with the $\mathcal{D}_{|0\rangle^{\otimes n}}$ training set. This result is consistent with the observations from previous sections. The difference in performance between the two training sets is apparent in both initialization conditions. Compared to the 4-qubit results, the differences in ROC curves between the two training sets are more significant in the 9-qubit case. This increase in separation can be attributed to the larger difference in the HNN outputs on trivial states observed in the previous section.

Conclusion

Based on the experiment results of this work, it is observed that the HNNs trained with the training set $\mathcal{D}_{|0\rangle^{\otimes n}}$ only learn to recognize the features of the $|0\rangle$ state, instead of the topological state. On the other hand, the HNNs trained with the training set $\mathcal{D}_{I/2^n}$ performs better on a more comprehensive verification set by achieving lower error probabilities in the binary classification task. Inferring based on the experiment results of this work, the machine learning model trained in [4] might suffer from the same drawback that the model only learns the features of the trivial quantum phase, as a similar training set to $\mathcal{D}_{|0\rangle^{\otimes n}}$ was used.

This work represents the state-of-the-art of the application of quantum neural networks on 2D systems. The 9-qubit surface code model exhibits properties characteristic of 2D systems, such as topological order, despite being implemented on a 1D chain for practical reasons. The topological entanglement entropy [29] supports this 2D nature, as it reflects the non-local correlations unique to topological phases. 2D models are particularly interesting because they are computationally challenging for classical simulations due to the exponential growth of the Hilbert space with system size. Quantum computers, on the other hand, can provide a computational advantage in investigating these systems, as indicated by the successful training of HNNs in this work.

During the planning stage of this work, the advantages of using quantum circuits for phase recognition were thoroughly discussed; however, quantifiable evidence has yet to be provided. Potential methods for quantitative evaluation include analyzing sample complexity to assess efficiency and the area under the ROC curves to evaluate accuracy.

There exists potential improvement for the HNN architecture and training procedure. Instead of relying solely on the measurement in the Z-basis, Pauli-6 POVMs are proposed as an alternative in [30] and can be potentially implemented in the HNN experiment. Besides, the training set can be expanded to include a broader range of quantum states, such as surface code ground states perturbed with single-qubit errors or thermal decay. It is also possible to apply the HNN architecture to more general models, such as the cluster-Ising model. To enhance the performance of the HNN by further leveraging the power of classical machine learning, the classical post-processing unit could be replaced by a neural network in future studies. The detection of the phase boundary remains an open question, as it is unclear whether the HNN can learn to predict the phase boundary using a training set that includes only the two extreme points ($|\psi(0)\rangle$ and $|\psi(\infty)\rangle$). This can be explored by incorporating

additional states into the training set to assist the HNN to recognize the phase transition boundary.

Through this master thesis project, I got a firsthand taste of how actual quantum experiments are carried out. It's nothing like the speedy pace of lecture slides. Pushing the scientific frontier forward is a slow, meticulous process, and I now have a much deeper appreciation for the hard work and persistence it takes to make even the smallest breakthroughs. Thankfully, I had the best possible guide, Colin Scarato. Colin's countless pieces of valuable advice on both science and how to work efficiently have been absolutely invaluable and will stick with me for life. I'm also deeply grateful to Prof. Andreas Wallraff for giving me the opportunity to carry out my master thesis in Qudev, one of the world's leading groups in superconducting qubits research. And finally, to everyone in the group: thank you for making these nine months so memorable, both scientifically and personally. It's been a truly wonderful experience.

Last but certainly not least, I want to thank my parents, Xuehui Wang and Zhihai Wang, for their constant support not only during my master studies, but also throughout my life.

Bibliography

[1] Juan Carrasquilla and Roger G. Melko. Machine learning phases of matter. *Nature Physics*, 13:431, February 2017.

[2] Frank Pollmann and Ari M. Turner. Detection of symmetry-protected topological phases in one dimension. *Phys. Rev. B.*, 86:125441, 2012.

[3] M. Gong, H. Huang, S. Wang, C. Guo, S. Li, Y. Wu, Q. Zhu, Y. Zhao, S. Guo, H. Qian, Y. Ye, C. Zha, F. Chen, C. Ying, J. Yu, D. Fan, D. Wu, H. Su, H. Deng, H. Rong, K. Zhang, S. Cao, J. Lin, Y. Xu, L. Sun, C. Guo, N. Li, F. Liang, A. Sakurai, K. Nemoto, W. J. Munro, Y. Huo, C. Lu, C. Peng, X. Zhu, and J. Pan. Quantum neuronal sensing of quantum many-body states on a 61-qubit programmable superconducting processor. *arXiv:2201.05957*, 2022.

[4] H. Huang, R. Kueng, G. Torlai, V. V. Albert, and J. Preskill. Provably efficient machine learning for quantum many-body problems. *arXiv:2106.12627*, 2021.

[5] Iris Cong, Soonwon Choi, and Mikhail D Lukin. Quantum convolutional neural networks. *Nature Physics*, pages 1–6, 2019.

[6] J. Herrmann, S. M. Llima, A. Remm, P. Zapletal, N. A. McMahon, C. Scarato, F. Swiadek, C. K. Andersen, C. Hellings, S. Krinner, N. Lacroix, S. Lazar, M. Kerschbaum, D. C. Zanuz, G. J. Norris, M. J. Hartmann, A. Wallraff, and C. Eichler. Realizing quantum convolutional neural networks on a superconducting quantum processor to recognize quantum phases. *Nature Comm.*, 13:4144, 2022.

[7] Leon C. Sander, Nathan A. McMahon, Petr Zapletal, and Michael J. Hartmann. Quantum convolutional neural network for phase recognition in two dimensions. 2024.

[8] Ludwig Nützel, Alexander Gresch, Lukas Hehn, Lucas Marti, Robert Freund, Alex Steiner, Christian D. Marciniak, Timo Eckstein, Nina Stockinger, Stefan Wolf, Thomas Monz, Michael Kühn, and Michael J. Hartmann. Solving an industrially relevant quantum chemistry problem on quantum hardware. 2024.

[9] Xiaoxuan Pan, Zhide Lu, Weiting Wang, Ziyue Hua, Yifang Xu, Weikang Li, Weizhou Cai, Xuegang Li, Haiyan Wang, Yi-Pu Song, Chang-Ling Zou, Dong-Ling Deng, and Luyan Sun. Deep quantum neural networks equipped with backpropagation on a superconducting processor. 2022.

[10] Wikipedia contributors. Eigenstate thermalization hypothesis — Wikipedia, the free encyclopedia. `https://en.wikipedia.org/w/index.php?title=Eigenstate_thermalization_hypothesis&oldid=1193767723`, 2024. [Online; accessed 6-January-2025].

[11] Xiaoxuan Pan, Xi Cao, Weiting Wang, Ziyue Hua, Weizhou Cai, Xuegang Li, Haiyan Wang, Jiaqi Hu, Yipu Song, Dong-Ling Deng, Chang-Ling Zou, Re-Bing Wu, and Luyan Sun. Experimental quantum end-to-end learning on a superconducting processor. *npj Quantum Information*, 9(1):18, Mar 2023.

[12] Aleksei Tolstobrov, Gleb Fedorov, Shtefan Sanduleanu, Shamil Kadyrmetov, Andrei Vasenin, Aleksey Bolgar, Daria Kalacheva, Viktor Lubsanov, Aleksandr Dorogov, Julia Zotova, Peter Shlykov, Aleksei Dmitriev, Konstantin Tikhonov, and Oleg V. Astafiev. Hybrid quantum learning with data reuploading on a small-scale superconducting quantum simulator. *Physical Review A*, 109(1):012411, January 2024.

[13] Jennifer R. Glick, Tanvi P. Gujarati, Antonio D. Córcoles, Youngseok Kim, Abhinav Kandala, Jay M. Gambetta, and Kristan Temme. Covariant quantum kernels for data with group structure. *Nature Physics*, Jan 2024.

[14] Tak Hur, Israel F. Araujo, and Daniel K. Park. Neural quantum embedding: Pushing the limits of quantum supervised learning. *arXiv:2311.11412*, November 2023.

[15] Thomas E O'Brien, G Anselmetti, Fotios Gkritsis, VE Elfving, Stefano Polla, William J Huggins, Oumarou Oumarou, Kostyantyn Kechedzhi, Dmitry Abanin, Rajeev Acharya, et al. Purification-based quantum error mitigation of pair-correlated electron simulations. *Nature Physics*, 19(12):1787–1792, 2023.

[16] Julien Vidal, Sébastien Dusuel, and Kai Phillip Schmidt. Low-energy effective theory of the toric code model in a parallel magnetic field. *Physical Review B—Condensed Matter and Materials Physics*, 79(3):033109, 2009.

[17] Markus Hoffmann. Training hybrid quantum-classical neural networks for the recognition of topological quantum phases. *Master's thesis*.

[18] Sébastien Dusuel, Michael Kamfor, Román Orús, Kai Phillip Schmidt, and Julien Vidal. Robustness of a perturbed topological phase. *Physical review letters*, 106(10):107203, 2011.

[19] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A*, 86:032324, Sep 2012.

[20] A. Hoffmann, S. Ramanathan, J. Grollier, A. D. Kent, M. Rozenberg, I. K. Schuller, O. Shpyrko, R. Dynes, Y. Fainman, A. Frano, E. E. Fullerton, G. Galli, V. Lomakin, S. P. Ong, A. K. Petford-Long, J. A. Schuller, M. D. Stiles, Y. Takamura, and Y. Zhu. Quantum materials for energy-efficient neuromorphic computing. *arXiv:2204.01832*, 2022.

[21] Colin Scarato, Kilian Hanke, Ants Remm, Stefania Lazar, Nathan Lacroix, Dante Colao Zanuz, Alexander Flasby, Andreas Wallraff, and Christoph Hellings. A continuous set of controlled-phase gates parameterized by an idle time. *in preparation*.

[22] Kilian Hanke. On-chip training of a quantum convolutional neural network using a continuous set of controlled-phase gates. Master's thesis, ETH Zurich, 2023.

[23] Anna Dawid, Julian Arnold, Borja Requena, Alexander Gresch, Marcin Płodzień, Kaelan Donatella, Kim A. Nicoli, Paolo Stornati, Rouven Koch, Miriam Büttner, Robert Okuła, Gorka Muñoz-Gil, Rodrigo A. Vargas-Hernández, Alba Cervera-Lierta, Juan Carrasquilla, Vedran Dunjko, Marylou Gabrié, Patrick Huembeli, Evert van Nieuwenburg, Filippo Vicentini, Lei Wang, Sebastian J. Wetzel, Giuseppe Carleo, Eliška Greplová, Roman Krems, Florian Marquardt, Michał Tomza, Maciej Lewenstein, and Alexandre Dauphin. Modern applications of machine learning in quantum sciences. *arXiv:2204.04198*, 2022.

[24] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. 2017.

[25] Sebastian Krinner, Nathan Lacroix, Ants Remm, Agustin Di Paolo, Elie Genois, Catherine Leroux, Christoph Hellings, Stefania Lazar, Francois Swiadek, Johannes Herrmann, Graham J. Norris, Christian Kraglund Andersen, Markus Müller, Alexandre Blais, Christopher Eichler, and Andreas Wallraff. Realizing repeated quantum error correction in a distance-three surface code. *Nature*, 605(7911):669–674, May 2022.

[26] D. Ristè, C. C. Bultink, K. W. Lehnert, and L. DiCarlo. Feedback control of a solid-state qubit using high-fidelity projective measurement. *Phys. Rev. Lett.*, 109:240502, Dec 2012.

[27] Yves Salathé, Philipp Kurpiers, Thomas Karg, Christian Lang, Christian Kraglund Andersen, Abdulkadir Akin, Sebastian Krinner, Christopher Eichler, and Andreas Wallraff. Low-latency digital signal processing for feedback and feedforward in quantum computing and communication. *Phys. Rev. Appl.*, 9:034011, Mar 2018.

[28] Ara Ghukasyan, Jack S. Baker, Oktay Goktas, Juan Carrasquilla, and Santosh Kumar Radha. Quantum-classical multiple kernel learning. 2023.

[29] K. J. Satzinger, Y.-J Liu, A. Smith, C. Knapp, M. Newman, C. Jones, Z. Chen, C. Quintana, X. Mi, A. Dunsworth, C. Gidney, I. Aleiner, F. Arute, K. Arya, J. Atalaya, R. Babbush, J. C. Bardin, R. Barends, J. Basso, A. Bengtsson, A. Bilmes, M. Broughton, B. B. Buckley, D. A. Buell, B. Burkett, N. Bushnell, B. Chiaro, R. Collins, W. Courtney, S. Demura, A. R. Derk, D. Eppens, C. Erickson, L. Faoro, E. Farhi, A. G. Fowler, B. Foxen, M. Giustina, A. Greene, J. A. Gross, M. P. Harrigan, S. D. Harrington, J. Hilton, S. Hong, T. Huang, W. J. Huggins, L. B. Ioffe, S. V. Isakov, E. Jeffrey, Z. Jiang, D. Kafri, K. Kechedzhi, T. Khattar, S. Kim, P. V. Klimov, A. N. Korotkov, F. Kostritsa, D. Landhuis, P. Laptev, A. Locharla, E. Lucero, O. Martin, J. R. McClean, M. McEwen, K. C. Miao, M. Mohseni, S. Montazeri, W. Mruczkiewicz, J. Mutus, O. Naaman, M. Neeley, C. Neill, M. Y. Niu, T. E. O'Brien, A. Opremcak, B. Pató,

A. Petukhov, N. C. Rubin, D. Sank, V. Shvarts, D. Strain, M. Szalay, B. Villalonga, T. C. White, Z. Yao, P. Yeh, J. Yoo, A. Zalcman, H. Neven, S. Boixo, A. Megrant, Y. Chen, J. Kelly, V. Smelyanskiy, A. Kitaev, M. Knap, F. Pollmann, and P. Roushan. Realizing topologically ordered states on a quantum processor. *Science*, 374(6572):1237–1241, 2021.

[30] Nicolas Sadoune, Ivan Pogorelov, Claire L. Edmunds, Giuliano Giudici, Giacomo Giudice, Christian D. Marciniak, Martin Ringbauer, Thomas Monz, and Lode Pollet. Learning symmetry-protected topological order from trapped-ion experiments. 2024.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Realizing Hybrid Quantum Neural Networks on a 9-qubit Surface Code to Recognize Topological Quantum Phases

Authored by (in block letters):

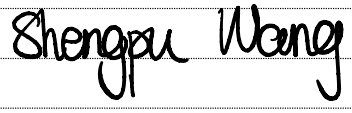
For papers written by groups the names of all authors are required.

Name(s):	**First name(s):**
Wang	Shengpu

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the 'Citation etiquette' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date	**Signature(s)**
20.12.2025	

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.